A

Project Report

on

RASPBERRY PI-ENABLED MULTI-MODAL AI SYSTEM FOR DEEPFAKE DETECTION IN MULTIMEDIA

Submitted in partial fulfillment of the requirements
for the award of the Degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

P. PAVAN KALYAN REDDY	23BFA04L34
P. SRAVANTHI	22BFA04317
T. BABY SHALINI	22BFA04324
N. BHAVYA	22BFA04298
D. AJAY	23BFA04L20

Under the guidance of
Dr. E. ANANT SHANKAR, Ph.D.,
Associate Professor

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
SRI VENKATESWARA COLLEGE OF ENGINEERING
(AUTONOMOUS)
(Accredited by NBA & NAAC with 'A' grade, Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)
Karakambadi Road, TIRUPATI – 517507

2022-2026

SRI VENKATESWARA COLLEGE OF ENGINEERING (AUTONOMOUS)

(Accredited by NBA& NAACwith 'A' Grade, Approved by AICTE, New Delhi &Affiliated to JNTUA, Ananthapuramu)
Karakambadi Road, TIRUPATI – 517507

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that the project report entitled **"RASPBERRY PI-ENABLED MULTI-MODAL AI SYSTEM FOR DEEPFAKE DETECTION IN MULTIMEDIA"** *is a bonafide record of the project work done and submitted by*

P. PAVAN KALYAN REDDY	**23BFA04L34**
P. SRAVANTHI	**22BFA04317**
T. BABY SHALINI	**22BFA04324**
N. BHAVYA	**22BFA04298**
D. AJAY	**23BFA04L20**

for the partial fulfillment of the requirements for the award of B.Tech. Degree in **ELECTRONICS AND COMMUNICATION ENGINEERING**, *JNT University Anantapur, Ananthapuramu.*

GUIDE **Head of the Department**

External Viva-Voce Exam Held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

We hereby declare that the project report entitled **"RASPBERRY PI-ENABLED MULTI-MODAL AI SYSTEM FOR DEEPFAKE DETECTION IN MULTIMEDIA"** submitted to the department of **ELECTRONICS AND COMMUNICATION ENGINEERING** in partial fulfillment of requirements for the award of the degree of **BACHELOR OF TECHNOLOGY**. This project is the result of our own effort and that it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above.

P. PAVAN KALYAN REDDY	23BFA04L34
P. SRAVANTHI	22BFA04317
T. BABY SHALINI	22BFA04324
N. BHAVYA	22BFA04298
D. AJAY	23BFA04L20

ACKNOWLEDGEMENT

We are thankful to our guide **Dr. E. Anant Shankar**, Associate Professor, ECE Department for his valuable guidance and encouragement. His helping attitude and suggestions have helped us in the successful completion of the project.

We would like to express our gratefulness and sincere thanks to **Dr. D. Srinivasulu Reddy**, Head of the Department of **Electronics and Communication Engineering**, for his kind help and encouragement during the course of our study and in the successful completion of the project work.

We have great pleasure in expressing our hearty thanks to our beloved Principal **Dr. Vijaya Gunturu** for his constant encouragement and support in all aspects to complete this project.

Successful completion of any project cannot be done without proper support and encouragement. We sincerely thank the **Management** for providing all the necessary facilities during the course of study.

We would like to thank our parents and friends, who have the greatest contributions in all our achievements, for the great care and blessings in making us successful in all our endeavors.

P. PAVAN KALYAN REDDY	23BFA04L34
P. SRAVANTHI	22BFA04317
T. BABY SHALINI	22BFA04324
N. BHAVYA	22BFA04298
D. AJAY	23BFA04L20

ABSTRACT

Deepfakes are digitally manipulated multimedia content such as images, videos, and audio that can appear highly realistic and are often difficult to detect using human observation. The rapid advancement of artificial intelligence technologies has made it possible to generate synthetic media that closely resembles authentic content. This project presents a real-time deepfake detection system implemented using a Raspberry Pi platform that can analyze multimedia inputs obtained from a live camera or from stored media files. Inside the Raspberry-Pi, lightweight deep learning models are used to detect manipulated content efficiently. MobileNetV2 models are used for detecting deepfake images and video frames, while a Light-Convolutional Neural Network (LCNN) is used for detecting synthetic audio signals. The system performs preprocessing operations such as image resizing, video frame extraction, and Mel spectrogram generation for audio analysis before feeding the data into the trained models. The models are converted into TensorFlow Lite format and deployed on the Raspberry Pi to enable efficient edge-based inference. The system analyzes the input media and predicts whether the content is real or manipulated. The final output is displayed on the monitor and LCD display connected to the Raspberry Pi, and a buzzer alert is activated when fake media is detected. This system provides a portable, low-cost, and efficient solution for detecting deepfake multimedia content using an embedded artificial intelligence platform.

CONTENTS

	Abstract	**i**
	Table of contents	**ii**
	List of figures	**v**
	List of tables	**vi**
	Abbreviations	**vii**
Chapter No.	**Description**	**Page No.**
1	**INTRODUCTION**	**1-11**
	1.1 Introduction	1-2
	1.2 Background of Artificial Intelligence	3-4
	1.3 Deepfake Technology	4-6
	1.3.1 Types of Deepfake Media	6
	1.4 Impact of Deepfake Technology	7-8
	1.5 Challenges in Deepfake Detection	8-9
	1.6 Problem Statement	9-10
	1.7 Motivation for Deepfake Detection	10
	1.8 Objectives of the Work	11
2	**LITERATURE SURVEY**	**12-15**
	2.1 Exposing Fake Faces with Content and Trace Features	12
	2.2 Deepfake Detection: A Systematic Literature Review	12-13
	2.3 Deep Learning for Deepfakes Creation and Detection: A Survey	13
	2.4 Comparative Analysis of Deepfake Detection Methods	13
	2.5 Deepfake Detection Using Deep Learning Techniques: A Review	14
	2.6 Hybrid Xception-LSTM Model with Channel and Spatial Attention	14
	2.7 Development of Deepfake Detection Techniques for Protecting Multimedia Information	15

	2.8 Study on Deep Fake Face Detection Techniques	15
3	**EXISTING METHOD**	**16-22**
	3.1 Introduction	16-17
	3.2 Existing Deepfake Detection Approaches	17-20
	3.2.1 CNN-Based Spatial Feature Detection	17
	3.2.2 Handcrafted Feature-Based Detection Methods	18
	3.2.3 Single Deep Learning Model-Based Detection	18-19
	3.2.4 Frame-by-Frame Video Classification	19
	3.2.5 Cloud-Based Deepfake Detection Systems	20
	3.3 Limitations of Existing Deepfake Detection Systems	20-21
	3.4 Need for an Improved Deepfake Detection System	22
4	**PROPOSED METHOD**	**23-46**
	4.1 Introduction	23
	4.2 Scope of the Study	24
	4.3 Overview of the Proposed System	24-26
	4.4 System Architecture	26-29
	4.4.1 Input Media Module	27
	4.4.2 Preprocessing Module	27-28
	4.4.3 Deepfake Detection Models	28-29
	4.4.4 TensorFlow Lite Inference Module	29
	4.4.5 Raspberry Pi Processing Unit	29
	4.4.6 Output Module	29-30
	4.5 Block Diagram of the Proposed System	30-31
	4.6 System Workflow	31-36
	4.7 Deepfake Detection Algorithm	36
	4.8 Dataset Description	36-38
	4.8.1 Image Dataset Structure	37
	4.8.2 Video Dataset Structure	37
	4.8.3 Audio Dataset Structure	38
	4.9 Deepfake Detection Models	38-45
	4.9.1 MobileNetV2 Model for Image Detection	38-41
	4.9.2 MobileNetV2 Model for Video Frame Detection	41-42

	4.9.3 LCNN Model for Audio Deepfake Detection	42-45
	4.10 Advantages of the Proposed System	45
	4.11 Applications of the Proposed System	46
5	**HARDWARE AND SOFTWARE REQUIREMENTS**	**47-58**
	5.1 Introduction	47
	5.2 Hardware Components	48-54
	5.2.1 Raspberry Pi 4 Model B	48-50
	5.2.2 USB Webcam	50-51
	5.2.3 LCD Display	51-52
	5.2.4 Buzzer Module	52-23
	5.2.5 Memory Card	53
	5.2.6 Power Supply Adapter	53-54
	5.3 Software Components	54-58
	5.3.1 Raspberry Pi OS	54-55
	5.3.2 Python Programming Language	55
	5.3.3 OpenCV Library	55-56
	5.3.4 TensorFlow Lite Framework	56
	5.3.5 VNC Viewer	56-57
	5.3.6 Advanced IP Scanner	57
	5.3.7 Software Implementation Workflow	57-58
	5.3.8 Model Deployment on Raspberry Pi	58
6	**EXPERIMENTAL RESULTS**	**59-66**
	6.1 Introduction	59
	6.2 Experimental Results and Output Analysis	59-63
	6.3 Performance Evaluation Based on Classification Metrics	63-66
	6.3.1 Evaluation Metrics Definition	64
	6.3.2 Computation of Results	65
	6.3.3 Performance Analysis	65
	6.3.4 Key Observations	66
7	**CONCLUSION AND FUTURE SCOPE**	**67**
	REFERENCES	**68-69**
	APPENDIX	**70-80**

LIST OF FIGURES

Fig. No.	Figure Name	Page No.
1.1	Example of Real and Deepfake Media	2
1.2	Types of Deepfake Media	6
1.3	Impact of Deepfake Technology in Digital Media	8
4.1	System Architecture of the Proposed System	26
4.2	Block Diagram of Proposed System	30
4.3	System Workflow of the Proposed System	32
4.4	Architecture of MobileNetV2	39
4.5	LCNN Architecture for Audio Deepfake Detection	43
5.1	Raspberry Pi 4 Model B	49
5.2	USB Webcam used for capturing image	50
5.3	16×2 LCD Display	52
5.4	Buzzer	52
6.1	Detection of Fake Video Frame	59
6.2	Detection of Real Video Frame	60
6.3	LCD Display Showing Final Detection Output	60
6.4	LCD Output Display Showing Real Classification	61
6.5	LCD Output Display Showing Fake Classification	62
6.6	Audio-Based Detection Result Fake Output	62
6.7	Audio-Based Detection Result Real Output	63

LIST OF TABLES

Table No.	Table Name	Page No.
1.1	Evolution of Artificial Intelligence	3
1.2	Risks Associated with Deepfake Technology	7
4.1	Image Dataset Used in the Proposed System	37
4.2	Video Dataset Used in the Proposed System	37
4.3	Audio Dataset Used in the Proposed System	37
6.1	Performance Metrics of Proposed System	63

ABBREVIATIONS

AI	-	Artificial Intelligence
CNN	-	Convolutional Neural Network
DL	-	Deep Learning
GAN	-	Generative Adversarial Network
GPIO	-	General Purpose Input Output
GPU	-	Graphics Processing Unit
IP	-	Internet Protocol
LCD	-	Liquid Crystal Display
LCNN	-	Light Convolutional Neural Network
ML	-	Machine Learning
OpenCV	-	Open Source Computer Vision Library
OS	-	Operating System
SVM	-	Support Vector Machine
TFLite	-	TensorFlow Lite
VNC	-	Virtual Network Computing

CHAPTER 1

INTRODUCTION

1.1 Introduction

Digital media has become one of the most dominant forms of communication in modern society. Images, videos, and audio recordings are widely used across social media platforms, entertainment industries, educational environments, and communication systems. The rapid growth of digital technologies has enabled individuals to create and distribute multimedia content easily through the internet. While these advancements have improved communication and information sharing, they have also introduced new challenges regarding the authenticity and reliability of digital content.

In recent years, artificial intelligence (AI) and deep learning technologies have significantly transformed the way multimedia content is generated and manipulated. One of the most notable developments in this area is the emergence of deepfake technology, which uses deep learning algorithms to create highly realistic manipulated media. Deepfake techniques can modify facial expressions in images, replace faces in videos, or generate synthetic speech that mimics real human voices. These manipulations are often so realistic that they become difficult to detect using traditional methods or human observation.

Deepfake technology has rapidly gained attention because of its potential to both benefit and harm society. On the positive side, deepfake techniques can be used in areas such as filmmaking, gaming, virtual reality, and digital content creation. For example, film producers can recreate historical characters or modify scenes without requiring additional filming. However, the same technology can also be used maliciously to create misleading content, impersonate individuals, or spread misinformation.

The increasing availability of deepfake generation tools has made it possible for individuals with limited technical expertise to create manipulated media. As a result, the authenticity of digital content shared on online platforms has become a major concern. Deepfake videos and audio recordings can be used to spread false information, manipulate public opinion, or damage an individual's reputation. These risks highlight the importance of developing reliable systems capable of detecting manipulated multimedia content.

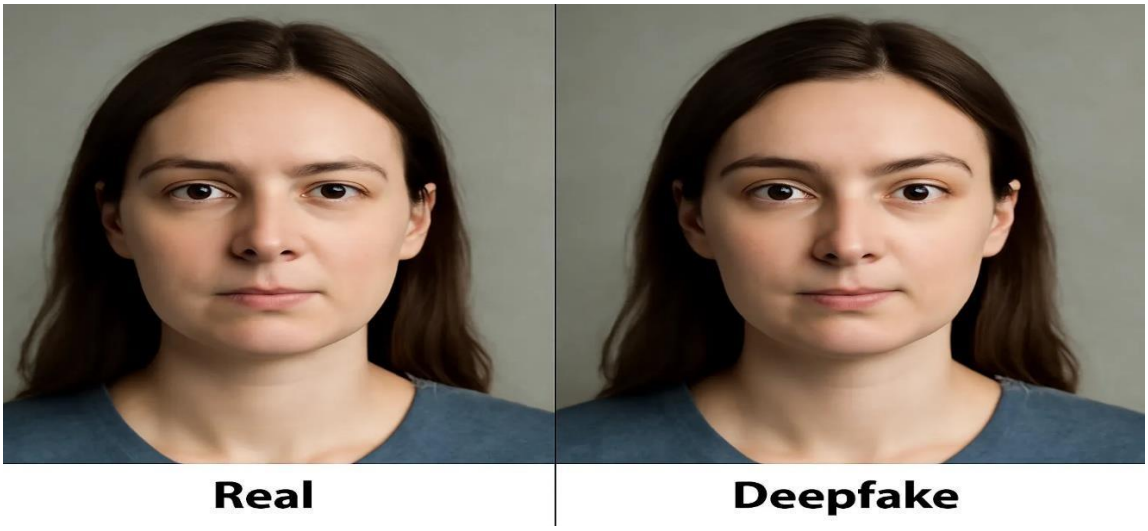

Fig. 1.1: Example of Real and Deepfake Media

Figure 1.1 illustrates a comparison between real and deepfake images, demonstrating the visual variations that help in identifying manipulated media.

Deepfake detection has therefore become an important research area in artificial intelligence and multimedia security. Researchers are developing automated systems that analyze multimedia data and identify patterns indicating manipulation. These systems often rely on machine learning and deep learning algorithms capable of detecting subtle inconsistencies that may not be visible to the human eye.

The development of automated deepfake detection systems is essential for maintaining trust in digital media. As deepfake generation techniques continue to evolve, detection methods must also improve in order to ensure that manipulated content can be identified accurately and efficiently.

1.2 Background of Artificial Intelligence

Artificial intelligence is a field of computer science that focuses on creating systems capable of performing tasks that normally require human intelligence. These tasks include learning from experience, recognizing patterns, understanding language, solving problems, and making decisions.

The concept of artificial intelligence has evolved significantly over the past several decades. Early AI systems relied on rule-based approaches in which computers followed predefined instructions to perform specific tasks. These systems were limited because they lacked the ability to learn from data or adapt to new situations.

With the advancement of computing technologies and the availability of large datasets, artificial intelligence began to shift toward machine learning, a technique that allows computers to learn patterns from data rather than relying solely on predefined rules. Machine learning algorithms analyze large datasets to identify relationships between input data and expected outputs.

Table 1.1: Evolution of Artificial Intelligence

Period	Technology Stage	Key Characteristics
1950–1970	Early AI	Rule-based systems and symbolic reasoning
1980–1990	Machine Learning	Algorithms learned patterns from data
2000–2010	Data-driven AI	Large datasets enabled better training
2010–Present	Deep Learning	Neural networks improved multimedia analysis

Machine learning has enabled the development of many modern technologies, including:

- Recommendation systems used by online platforms
- Email spam filtering systems

- Fraud detection systems in financial institutions
- Image and speech recognition technologies

One of the most important advancements in artificial intelligence is deep learning, which is a specialized form of machine learning that uses artificial neural networks with multiple layers. Deep learning models are capable of processing complex data structures such as images, videos, and audio signals.

Deep learning has significantly improved the performance of many AI applications, including:

- Computer vision systems
- Speech recognition technologies
- Natural language processing systems
- Multimedia analysis applications

These advancements have made deep learning a key technology in the development of deepfake generation and detection systems.

1.3 Overview of Deepfake Technology

Deepfake technology refers to the use of deep learning algorithms to create or manipulate multimedia content in a way that makes it appear realistic. The term "deepfake" is derived from the combination of deep learning and fake media, indicating the use of neural networks to generate synthetic multimedia content.

Deepfake generation techniques typically rely on advanced neural network architectures capable of learning patterns from large datasets. These models analyze facial features, speech characteristics, and motion patterns to generate realistic synthetic media.

The development of deepfake technology has been largely driven by the advancement of Generative Adversarial Networks (GANs). GANs consist of two neural networks that are trained simultaneously:

- Generator, which produces synthetic media
- Discriminator, which evaluates whether the media is real or fake

During the training process, the generator attempts to produce realistic media that can deceive the discriminator, while the discriminator attempts to identify manipulated media. Through continuous training, both networks improve their performance, resulting in increasingly realistic outputs.

Deepfake technology has improved significantly over the past decade due to:

- Increased availability of large multimedia datasets
- Improvements in deep learning frameworks
- Availability of powerful graphics processing units (GPUs)
- Development of open-source AI tools

As a result, deepfake generation tools have become more accessible and easier to use.

1.3.1 Types of Deepfake Media

Deepfake manipulation can occur in multiple forms of multimedia data.

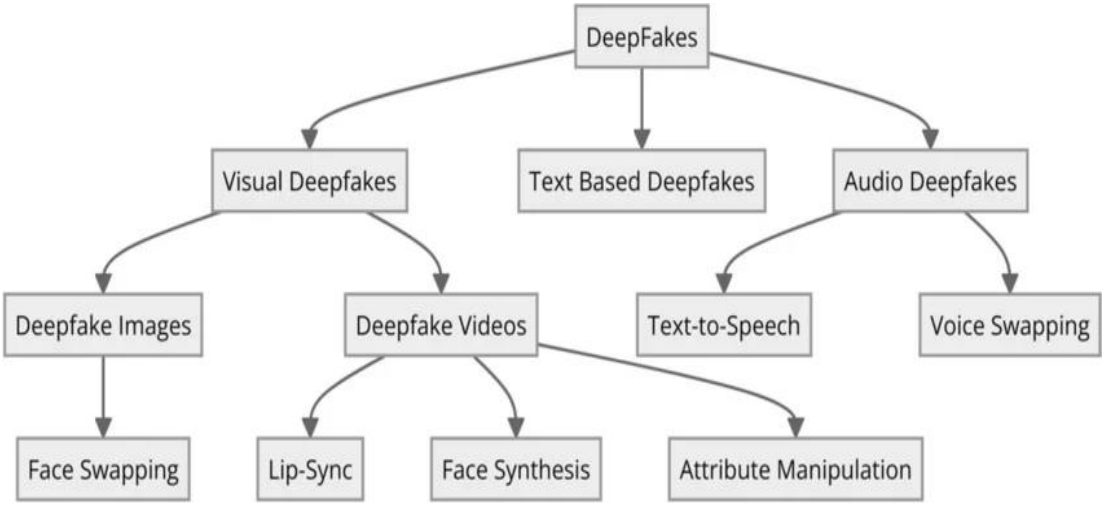

Fig. 1.2: Types of Deepfake Media

Figure 1.2 illustrates the different types of deepfake media, including visual, audio, and text-based manipulations along with their subcategories.

Image Deepfakes

Image deepfakes involve the manipulation or generation of images using deep learning models. These manipulations may include face swapping, facial expression editing, or the creation of entirely synthetic faces.

Video Deepfakes

Video deepfakes involve altering video frames to replace a person's face or modify facial expressions. These manipulations are often performed frame by frame using deep learning models that replicate facial movements.

Audio Deepfakes

Audio deepfakes involve generating synthetic speech using machine learning models capable of imitating a person's voice. These models analyze voice characteristics such as pitch, tone, and speech patterns to generate realistic audio recordings.

1.4 Impact of Deepfake Technology

The development of deepfake technology has had a significant impact on digital media and society. While the technology offers useful applications in entertainment and digital content creation, it also introduces serious risks related to misinformation and digital security.

Table 1.2: Risks Associated with Deepfake Technology

Risk Type	Description	Impact
Misinformation	Manipulated media used to spread false information	Misleads public opinion and creates confusion
Identity Impersonation	Use of deepfake technology to imitate a person's face or voice	Enables fraud and unauthorized communication
Financial Fraud	Synthetic media used to deceive individuals or organizations	Causes financial losses and security threats
Reputation Damage	Altered images or videos harming a person's credibility	Damages trust and public image

Deepfake media can be used to create misleading videos or audio recordings that appear authentic. These manipulations can influence public opinion, damage reputations, or create confusion regarding real-world events.

Figure 1.3 illustrates the various applications and misuse areas of deepfake technology in digital media, including celebrity pornography, election manipulation, social engineering, automated disinformation attacks, scams and hoaxes, financial fraud, and identity theft. These examples highlight how deepfake content can be exploited to manipulate information, deceive individuals, and create serious social and financial risks.

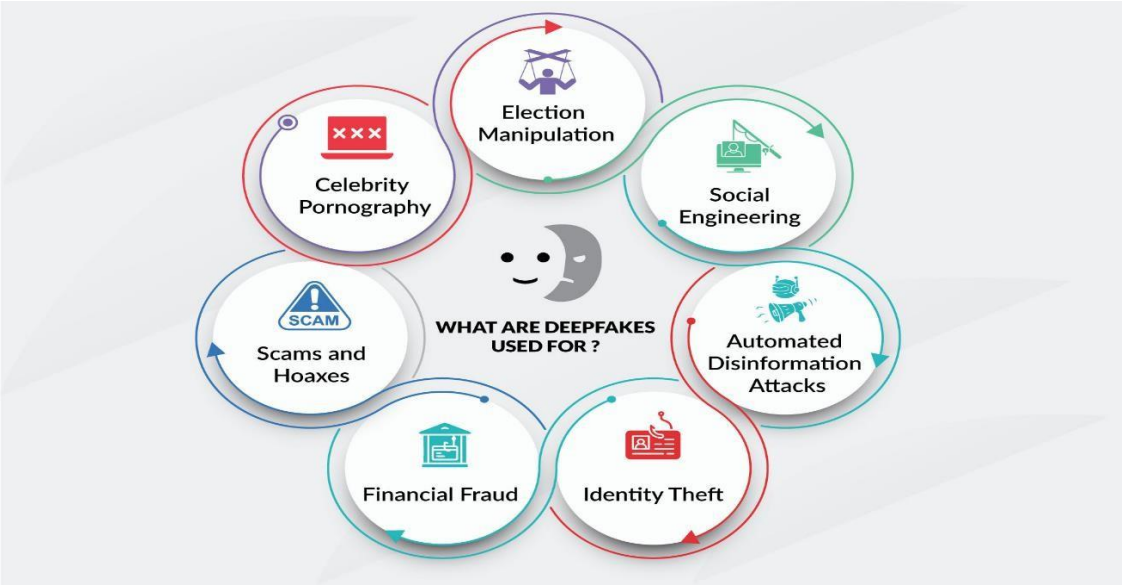

Fig. 1.3: Impact of Deepfake Technology in Digital Media

Some of the major concerns associated with deepfake technology include:

- Spread of misinformation through manipulated media
- Identity impersonation using voice or face cloning
- Digital fraud involving synthetic audio recordings
- Damage to individual reputations through manipulated videos

The rapid spread of deepfake media on social media platforms has increased the urgency of developing reliable detection systems.

1.5 Challenges in Deepfake Detection

Despite the progress made in developing detection systems, identifying deepfake media remains a challenging task. Modern deepfake generation techniques produce highly realistic outputs that can closely mimic genuine media.

One of the main challenges is the rapid evolution of deepfake generation algorithms. As detection methods improve, new generation techniques are developed that attempt to bypass existing detection systems.

Other challenges include:

- Lack of large labeled datasets for training detection models
- Variations in video quality and compression levels
- Difficulty detecting manipulations in low-resolution media
- Rapid spread of manipulated media on social networks

Addressing these challenges requires continuous research and development of more advanced detection techniques.

1.6 Problem Statement

The rapid development of artificial intelligence and deep learning technologies has enabled the creation of highly realistic manipulated multimedia content known as deepfakes. These deepfakes can modify images, videos, and audio recordings in such a way that they appear authentic to human observers. As deepfake generation techniques continue to improve, detecting manipulated media becomes increasingly difficult.

Deepfake content can be misused in many areas such as social media, online communication, and digital platforms. Fake multimedia content can spread misinformation, damage the reputation of individuals, and influence public opinion. Therefore, reliable systems are required to detect and identify manipulated media effectively.

However, several challenges exist in current deepfake detection systems:

• Deepfake media is becoming increasingly realistic and difficult to identify manually.

• Many detection systems focus only on one type of media such as images or videos.

• Existing systems often require powerful hardware and high computational resources.
• Cloud-based detection systems may introduce latency and privacy concerns.
• Real-time detection on lightweight embedded devices is still a major challenge.

To address these issues, there is a need for an efficient and portable deepfake detection system that can analyze multiple types of multimedia content. The system should be capable of detecting manipulated images, videos, and audio while operating on a low-power edge computing device.

This project aims to develop a **Raspberry Pi–enabled multi-modal deepfake detection system** that uses lightweight deep learning models to identify manipulated multimedia content efficiently.

1.7 Motivation for Deepfake Detection

The growing accessibility of deepfake generation tools has created an urgent need for reliable detection systems capable of identifying manipulated multimedia content. Human observers often find it difficult to distinguish between real and manipulated media, especially when deepfake techniques produce highly realistic outputs.

The motivation behind deepfake detection research is to:

- Preserve the authenticity of digital media
- Prevent the spread of misinformation
- Protect individuals from impersonation and fraud
- Improve digital media verification systems

Developing automated detection systems can help ensure that manipulated media can be identified before it spreads widely across online platforms.

1.8 Objectives of the Study

The primary objective of this study is to investigate techniques for detecting manipulated multimedia content using artificial intelligence methods.

The specific objectives include:

- Understanding the concept of deepfake technology and its impact on digital media
- Studying artificial intelligence techniques used for multimedia analysis
- Analyzing different types of deepfake manipulation in images, videos, and audio
- Exploring machine learning and deep learning techniques used in deepfake detection
- Investigating challenges associated with detecting manipulated media

CHAPTER 2

LITERATURE SURVEY

2.1 Exposing Fake Faces with Content and Trace Features (Eunji Kim & Sungzoon Cho, 2021)

Eunji Kim and Sungzoon Cho proposed a deepfake detection approach that combines content-based features and trace-based features to identify manipulated facial images. Content features focus on facial structures such as texture patterns, illumination inconsistencies, and facial landmark variations. Trace features analyze artifacts introduced during the manipulation process, including compression noise and blending inconsistencies. The authors used deep neural networks to extract these features and classify images as real or fake. Their experimental results demonstrated that combining both visual and forensic features significantly improves detection performance, particularly when dealing with compressed multimedia data. The study highlights the importance of analyzing multiple feature representations simultaneously in order to effectively detect deepfake manipulations [1].

2.2 Deepfake Detection: A Systematic Literature Review (Md Shohel Rana et al., 2022)

Rana and colleagues conducted a systematic review of deepfake detection techniques used in multimedia applications. The authors categorized detection methods into spatial-domain, temporal-domain, and frequency-domain approaches. Spatial-domain techniques analyze pixel-level inconsistencies present in manipulated images. Temporal-domain techniques focus on frame-to-frame irregularities in video sequences, while frequency-domain methods detect spectral artifacts introduced during deepfake generation. The study concluded that deep learning-based approaches outperform traditional machine learning techniques in detecting manipulated media.

However, the authors identified several challenges such as limited training datasets, compression artifacts, and variations in video quality that affect the reliability of deepfake detection systems [2].

2.3 Deep Learning for Deepfakes Creation and Detection: A Survey (Nguyen et al., 2022)

Nguyen and colleagues presented a comprehensive survey on deepfake generation and detection techniques based on deep learning architectures. The study discussed generative models such as Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and other neural network architectures used to create synthetic media. The authors also reviewed detection techniques including convolutional neural networks, frequency-domain analysis, and spatio-temporal detection models. Their analysis revealed that hybrid detection methods combining multiple feature extraction techniques achieve better performance compared to single-model approaches. The survey emphasized the importance of large-scale datasets and improved generalization strategies to build robust deepfake detection systems [3].

2.4 Comparative Analysis of Deepfake Detection Methods (Jerry John & Bismin Sherif, 2022)

Jerry John and Sherif analyzed several deepfake detection techniques and compared their performance across different datasets. Their study evaluated popular deep learning architectures such as VGGNet, ResNet, InceptionNet, and XceptionNet for detecting manipulated facial images. The authors observed that convolutional neural network models are effective in identifying image-level deepfake artifacts such as texture inconsistencies and pixel-level distortions. However, these models often struggle when detecting high-quality deepfake videos. To improve performance, the researchers suggested the use of synthetic data augmentation generated through GANs in order to increase the diversity of training datasets [4].

2.5 Deepfake Detection Using Deep Learning Techniques: A Review (Amala Mary & Anitha Edison, 2023)

Amala Mary and Anitha Edison conducted a detailed review of deep learning-based deepfake detection systems. Their study evaluated several neural network architectures including convolutional neural networks, residual networks, and transformer-based models. The authors observed that CNN-based models perform well in detecting facial manipulation artifacts in images, while hybrid architectures combining spatial and temporal features are more effective for video deepfake detection. The review also highlighted challenges such as dataset imbalance, compression noise, and generalization limitations that reduce model performance. The authors emphasized the need for lightweight deep learning architectures suitable for real-time deepfake detection applications [5].

2.6 Hybrid Xception-LSTM Model with Channel and Spatial Attention (Deepak Dagar & Vishwakarma, 2023)

Deepak Dagar and Vishwakarma proposed a hybrid deepfake detection architecture that combines the Xception convolutional neural network with Long Short-Term Memory (LSTM) networks. The Xception network extracts spatial features such as facial textures and blending artifacts present in manipulated images, while the LSTM network analyzes temporal inconsistencies across consecutive video frames. The authors also incorporated channel and spatial attention mechanisms to enhance the model’s ability to focus on important facial regions affected by manipulation. Experimental results demonstrated that the proposed hybrid architecture improves detection accuracy for video deepfakes compared to traditional CNN-based models [6].

2.7 Development of Deepfake Detection Techniques for Protecting Multimedia Information (Siva Rama Lingham et al., 2024)

Siva Rama Lingham and colleagues explored deepfake detection techniques aimed at protecting digital multimedia content from manipulation attacks. Their research compared traditional machine learning algorithms with deep learning-based approaches such as convolutional neural networks and transfer learning models. The authors observed that deep learning architectures significantly improve detection accuracy by automatically learning manipulation artifacts from large datasets. The study also emphasized the importance of lightweight and efficient detection systems capable of operating in real-time environments. Such systems are essential for protecting online platforms from the rapid spread of manipulated multimedia content [7].

2.8 Study on Deep Fake Face Detection Techniques (Mudit Arya et al., 2024)

Mudit Arya and co-authors conducted a detailed study on deepfake face detection techniques and the challenges associated with modern deepfake generation technologies. Their research examined several detection methods including CNN-based models, residual networks, and hybrid deep learning architectures. The authors highlighted that modern deepfake generation methods produce highly realistic outputs, making detection increasingly difficult. They suggested that multi-modal detection systems capable of analyzing images, videos, and audio simultaneously can significantly improve detection accuracy. Their findings support the development of AI-based detection systems that provide reliable protection against multimedia manipulation [8].

CHAPTER 3

EXISTING SYSTEM

3.1 Introduction

Deepfake technology has emerged as one of the most significant challenges in digital media authentication. With the advancement of artificial intelligence and deep learning techniques, it has become possible to generate highly realistic manipulated multimedia content. Deepfakes are typically created using deep learning models that can modify facial expressions, replace faces in videos, or synthesize human speech to imitate real individuals.

Although deepfake technology has useful applications in fields such as entertainment, film production, and digital content creation, it also poses serious threats to digital security and information authenticity. Manipulated multimedia content can be used to spread misinformation, impersonate individuals, or influence public opinion. As a result, detecting deepfake content has become an important research area in artificial intelligence and multimedia security.

To address this challenge, researchers have developed various deepfake detection systems that analyze multimedia content and identify manipulation artifacts. These detection methods rely on different computational techniques such as traditional machine learning algorithms, handcrafted feature analysis, and deep learning-based models. Each method attempts to identify patterns or inconsistencies that distinguish real media from manipulated media.

However, many existing deepfake detection systems have certain limitations. Some systems focus only on image-based detection, while others analyze video frames without considering temporal relationships. Additionally, several deep learning models require high computational resources, making them unsuitable for deployment on lightweight devices.

Understanding these existing detection techniques and their limitations is essential before designing improved deepfake detection frameworks.

3.2 Existing Deepfake Detection Approaches

Several approaches have been proposed for detecting deepfake content in multimedia data. These approaches differ in the type of features analyzed and the algorithms used to classify media as real or fake. The most commonly used deepfake detection techniques are described in the following sections.

3.2.1 CNN-Based Spatial Feature Detection

Convolutional Neural Networks (CNNs) are widely used in deepfake detection systems due to their ability to analyze spatial features in images and video frames. CNN models automatically extract patterns from input data and identify inconsistencies introduced during the deepfake generation process.

In deepfake detection systems, CNN models examine several visual characteristics present in facial regions. These include texture inconsistencies, abnormal pixel distributions, lighting mismatches, and boundary artifacts created when artificial faces are blended into original images or videos. CNNs process images or individual frames from videos and learn patterns associated with manipulated content.

The ability of CNNs to automatically learn features from raw image data has significantly improved deepfake detection accuracy compared to traditional feature-based approaches. Many modern detection systems use CNN architectures to classify images or frames as authentic or manipulated.

However, CNN-based systems often analyze each frame independently and primarily focus on spatial information. This means they may fail to capture temporal inconsistencies present in manipulated video sequences. Additionally, many CNN architectures require large computational resources, which can limit their deployment on low-power devices.

3.2.2 Handcrafted Feature-Based Detection Methods

Before the development of deep learning-based detection systems, many deepfake detection methods relied on handcrafted features extracted from multimedia data. These features were designed based on observable patterns in human facial movements and visual characteristics.

Common handcrafted features include eye-blink frequency, facial landmark positions, head movement patterns, and lip synchronization behavior. These features are extracted using image processing techniques and then provided as input to machine learning algorithms for classification.

Machine learning classifiers such as Support Vector Machines (SVM), Decision Trees, and Random Forest algorithms are commonly used to analyze these features. The classifier learns patterns from the extracted features and determines whether the media is real or manipulated.

Although handcrafted feature-based methods were useful in early deepfake detection research, they have several limitations. Modern deepfake generation techniques are capable of producing highly realistic facial movements and expressions. As a result, handcrafted feature-based detection systems often struggle to detect advanced deepfake manipulations.

3.2.3 Single Deep Learning Model-Based Detection

Many deepfake detection systems rely on a single deep learning architecture to analyze multimedia content. Popular neural network architectures used in these systems include XceptionNet, VGGNet, InceptionNet, and autoencoder-based models.

These models focus primarily on identifying spatial inconsistencies present in manipulated facial images. They analyze facial textures, blending artifacts, and abnormal pixel patterns to determine whether a face has been artificially generated or modified.

Single-model detection systems are relatively simple to implement and have shown good performance in image-based deepfake detection tasks. However, relying on a single neural network model may limit the system's ability to detect more complex manipulations.

Deepfake generation techniques continue to evolve, and modern deepfake videos may contain subtle artifacts that are difficult to detect using a single model. Therefore, many researchers are exploring more advanced detection frameworks that combine multiple feature extraction methods or use multi-modal detection strategies.

3.2.4 Frame-by-Frame Video Classification

Another common deepfake detection approach is frame-by-frame video classification. In this method, video sequences are divided into individual frames, and each frame is analyzed separately using image classification models.

This approach simplifies the detection process because image-based models can be directly applied to video frames. The classification results from individual frames are then combined to determine whether the entire video is real or manipulated.

However, frame-by-frame classification methods have several drawbacks. Since each frame is analyzed independently, the system ignores temporal relationships between consecutive frames. Deepfake videos often contain unnatural motion patterns or inconsistencies between frames that cannot be detected using independent frame analysis.

Furthermore, classification results may vary across frames, which can lead to inconsistent detection outcomes. These limitations reduce the reliability of frame-by-frame video classification in real-world deepfake detection applications.

3.2.5 Cloud-Based Deepfake Detection Systems

Some advanced deepfake detection systems rely on cloud computing platforms to perform computationally intensive analysis. In cloud-based detection systems, multimedia data is uploaded to remote servers where powerful deep learning models analyze the content.

Cloud computing allows detection systems to use high-performance computing resources and large neural network models that may not be feasible to run on local devices. This can improve detection accuracy in certain scenarios.

However, cloud-based detection systems introduce several practical challenges. Transferring multimedia data to remote servers requires a stable internet connection and can introduce delays due to network latency. Additionally, sending sensitive multimedia content to external servers raises privacy and security concerns.

Because of these limitations, cloud-based detection systems are not always suitable for real-time applications or environments where data privacy is important.

3.3 Limitations of Existing Deepfake Detection Systems

Although several deepfake detection techniques have been developed, many existing systems still face a number of limitations that affect their performance and practical deployment. These limitations arise due to the complexity of modern deepfake generation techniques, computational constraints, and the inability of some systems to analyze multiple types of multimedia data. The major limitations observed in existing deepfake detection systems are discussed below.

1. Limited Media Coverage

Most deepfake detection systems focus on detecting manipulations only in images or videos. Such systems are not designed to analyze multiple types of multimedia data simultaneously. As deepfake technology is now capable of generating manipulated images, videos, and audio recordings, single-media detection systems are often insufficient for real-world scenarios.

2. Reliance on Handcrafted Features

Earlier detection methods relied heavily on manually designed features such as facial landmarks, eye-blink rates, and texture patterns. These handcrafted features may work for simple manipulations but often fail when dealing with advanced deepfake techniques that produce highly realistic facial expressions and movements.

3. High Computational Requirements

Many deep learning-based detection models require powerful hardware resources such as GPUs and high-performance processors. These requirements increase system complexity and make it difficult to deploy such models on lightweight embedded devices.

4. Reduced Performance in Real-World Conditions

The performance of many detection systems decreases when multimedia data contains variations such as lighting changes, compression artifacts, camera noise, or fast motion. These real-world conditions make it more difficult for detection algorithms to accurately identify manipulated media.

5. Lack of Real-Time Detection Capability

Several deepfake detection systems are designed primarily for offline analysis or cloud- based processing. These systems may introduce delays and are not always suitable for real-time detection scenarios, especially when deployed on low-power devices.

3.4 Need for an Improved Deepfake Detection System

The increasing sophistication of deepfake generation technologies has created a strong need for improved detection systems. An effective deepfake detection framework should be capable of analyzing different types of multimedia data and identifying manipulation artifacts with high accuracy.

In addition, such systems should be computationally efficient and capable of operating on low-power hardware platforms. Lightweight detection models can enable real-time analysis of multimedia data without requiring high-performance computing infrastructure.

An improved detection system should also be able to handle variations in multimedia data such as lighting conditions, video compression, and camera quality. By combining advanced artificial intelligence techniques with efficient processing methods, it is possible to design deepfake detection systems that provide better performance and reliability.

CHAPTER 4

PROPOSED SYSTEM

4.1 Introduction

The rapid advancement of artificial intelligence and deep learning technologies has enabled the creation of highly realistic manipulated multimedia content known as deepfakes. Deepfake techniques use advanced neural networks to generate synthetic images, modify video frames, and create artificial speech that closely resembles real human voices. While these technologies offer innovative applications in areas such as digital entertainment, filmmaking, and virtual reality, they also pose serious threats related to misinformation, impersonation, digital fraud, and cybersecurity.

Traditional deepfake detection systems primarily focus on analyzing a single type of media such as images or videos. However, modern deepfake generation techniques can manipulate multiple forms of multimedia simultaneously, including images, videos, and audio recordings. Therefore, detection systems that rely on analyzing only one media type may fail to identify advanced deepfake manipulations.

To address this challenge, the proposed system introduces a multi-modal deepfake detection framework capable of analyzing images, videos, and audio recordings. The system integrates deep learning models with an embedded edge computing platform to perform real-time deepfake detection.

The proposed system is implemented using a Raspberry Pi-based edge AI platform, where trained deep learning models are deployed using TensorFlow Lite. The system processes multimedia inputs and determines whether the content is real or manipulated. The detection results are displayed on the monitor and LCD screen, and a buzzer alert is activated when fake media is detected.

This approach provides a lightweight and efficient deepfake detection solution capable of operating on low-power hardware devices while maintaining reliable performance.

4.2 Scope of the study

The scope of this project focuses on developing a system that can detect manipulated multimedia content using artificial intelligence techniques. The proposed system is designed to identify deepfake images, videos, and audio using lightweight deep learning models deployed on a Raspberry Pi platform.

The system analyzes multimedia inputs and determines whether the content is real or fake. The trained models are converted into TensorFlow Lite format so that they can run efficiently on the Raspberry Pi device. The results are displayed through a monitor and an LCD display, and a buzzer alert is activated when fake media is detected.

The main scope of this project includes the following:

- Detection of deepfake **images** using the MobileNetV2 model.
- Detection of deepfake **videos** by extracting and analyzing video frames.
- Detection of **synthetic audio** using the LCNN model.
- Deployment of trained models on **Raspberry Pi using TensorFlow Lite**.
- Processing multimedia input through **stored files or live camera input**.
- Displaying results on **monitor output window and LCD display**.
- Generating a **buzzer alert when manipulated media is detected**.

The project is implemented as a prototype system and uses a limited dataset for training and testing. Therefore, the system is mainly intended to demonstrate the feasibility of detecting deepfake multimedia content using a lightweight embedded platform.

4.3 Overview of the Proposed System

The proposed system is designed to detect deepfake manipulations across multiple types of multimedia content. Unlike traditional systems that focus on a single media type, the proposed system analyzes images, videos, and audio recordings.

The system uses trained deep learning models to analyze multimedia inputs and detect manipulation artifacts. The architecture is designed to operate efficiently on an edge computing platform such as Raspberry Pi.

The major components of the proposed system include:

1. Multimedia Input Module
2. Data Preprocessing Module
3. Feature Extraction Module
4. Deep Learning Detection Model
5. Raspberry Pi Processing Unit
6. Output Display Module
7. Alert Mechanism

The working principle of the system can be summarized as follows:

1. The user provides a multimedia input file such as an image, video, or audio recording.
2. The system identifies the type of media.
3. The input data is preprocessed to improve quality and consistency.
4. Features are extracted from the input media.
5. The deep learning model analyzes the extracted features.
6. The system classifies the media as Real or Fake.
7. The results are displayed on the monitor and LCD display.
8. A buzzer alert is triggered if fake media is detected.

This architecture enables efficient detection of deepfake manipulations across different multimedia formats.

4.4 System Architecture

The system architecture represents the structural design of the proposed Raspberry Pi–based multi-modal deepfake detection system. It illustrates the interaction between various system components responsible for detecting manipulated multimedia content. The architecture is designed to process different types of media inputs including images, videos, and audio recordings. By integrating lightweight deep learning models with an embedded computing platform, the system is capable of performing deepfake detection efficiently.

The architecture of the proposed system consists of multiple functional modules including input media acquisition, preprocessing, deepfake detection models, inference processing, and output modules. Each module performs a specific task in the detection pipeline. The overall architecture of the system is illustrated in **Fig 4.1**.

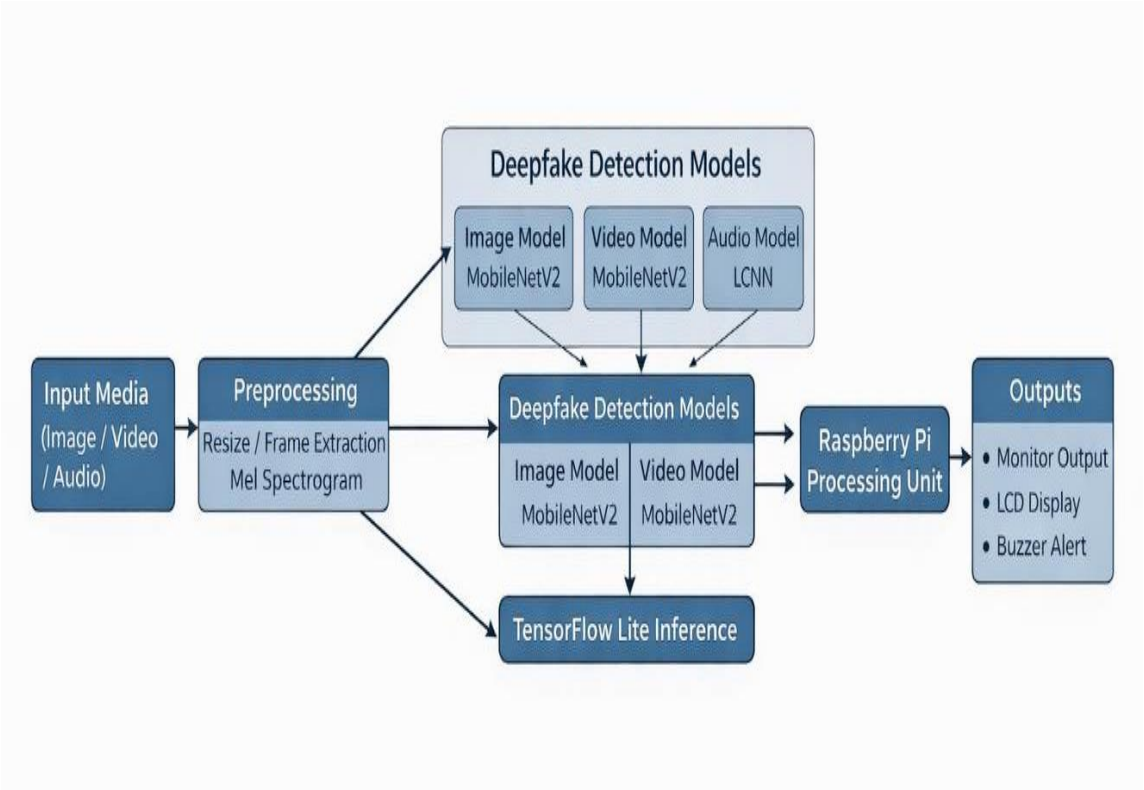

Fig 4.1 – System Architecture of proposed system

4.4.1 Input Media Module

The input media module is responsible for receiving multimedia data that will be analyzed by the system. The proposed system supports three different types of media inputs:

1. Image files
2. Video files
3. Audio recordings

The user provides the path of the media file to the system through the Python interface. Once the media file path is provided, the system loads the corresponding file and prepares it for preprocessing.

4.4.2 Preprocessing Module

The preprocessing module prepares the input media for deep learning analysis. Since different types of media have different formats, preprocessing operations are performed to standardize the input data.

The preprocessing operations include:

1. **Image preprocessing**
    - Image resizing to a fixed resolution
    - Normalization of pixel values
2. **Video preprocessing**
    - Extraction of individual frames from video
    - Frame resizing to match the input dimensions required by the deep learning model

3. **Audio preprocessing**
    - Conversion of audio signals into Mel spectrogram representations
    - Feature normalization for deep learning analysis

These preprocessing steps ensure that the multimedia data is compatible with the deep learning models used in the system.

4.4.3 Deepfake Detection Models

The deepfake detection module consists of three separate deep learning models designed to analyze different types of media. Each model is trained to identify manipulation artifacts present in deepfake content.

The models used in the system include:

1. **Image Deepfake Detection Model**

The image detection model is based on the MobileNetV2 architecture. MobileNetV2 is a lightweight convolutional neural network that is efficient for embedded systems such as Raspberry Pi. The model analyzes spatial features such as textures, pixel patterns, and facial inconsistencies to determine whether the image is real or manipulated.

2. **Video Deepfake Detection Model**

The video detection model also uses the MobileNetV2 architecture. In this case, the model processes frames extracted from the input video. By analyzing each frame, the system identifies visual inconsistencies that may indicate manipulated content.

3. **Audio Deepfake Detection Model**

The audio detection model uses the Light Convolutional Neural Network (LCNN) architecture. The LCNN model analyzes spectrogram representations of audio signals to detect anomalies in frequency patterns that indicate synthetic or manipulated speech.

These three models enable the system to perform multi-modal deepfake detection.

4.4.4 TensorFlow Lite Inference Module

The trained deep learning models are converted into TensorFlow Lite format to enable efficient execution on the Raspberry Pi platform. TensorFlow Lite is optimized for edge devices and provides faster inference while maintaining low computational requirements.

The inference module receives preprocessed media data and passes it through the appropriate deep learning model to generate a prediction indicating whether the media is real or fake.

4.4.5 Raspberry Pi Processing Unit

The Raspberry Pi acts as the central processing unit of the system. It coordinates all system operations including media loading, preprocessing, model inference, and result display.

The Raspberry Pi platform is chosen because it provides:

1. Low power consumption
2. Compact embedded computing capability
3. Compatibility with TensorFlow Lite
4. Ability to interface with hardware components such as LCD and buzzer

4.4.6 Output Module

After the deep learning model performs inference, the system generates the final detection result. The output module displays the classification results through different interfaces.

The outputs generated by the system include:

1. **Monitor Output**

The detection result is displayed in the Python output window connected to the monitor. The system indicates whether the analyzed media is real or fake.

2. **LCD Display**

The detection result is also displayed on the LCD screen connected to the Raspberry Pi. This provides a visual indication of the classification result.

3. **Buzzer Alert**

When manipulated media is detected, the buzzer is activated to alert the user. This provides an audible warning indicating that fake media has been identified.

4.5 Block Diagram of the Proposed System

Figure 4.2 illustrates the block diagram of the proposed system, showing the interconnection between the Raspberry Pi, input modules, and output devices.

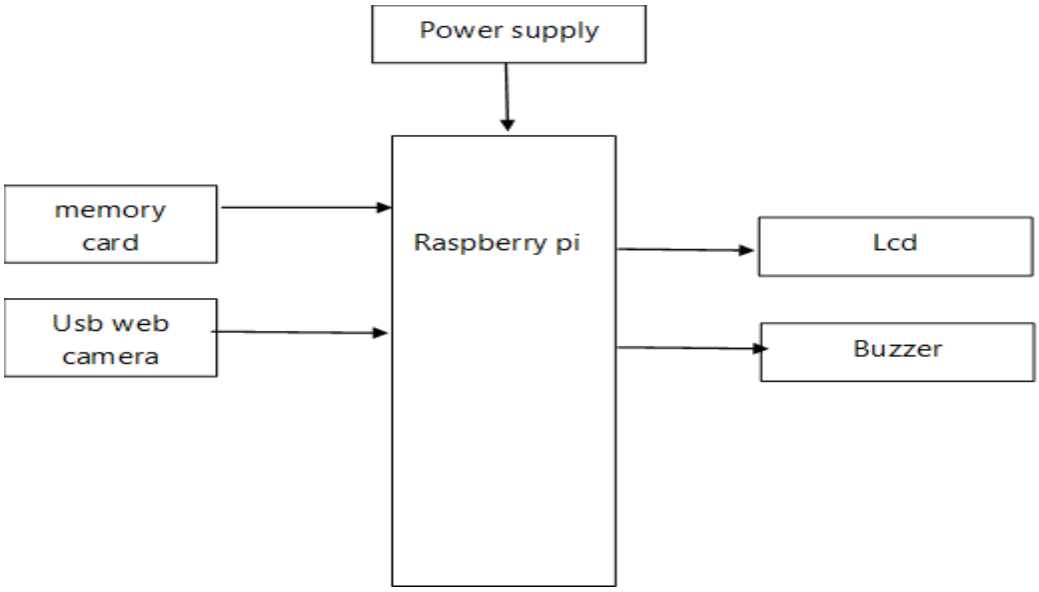

Fig 4.2 Block Diagram of Proposed System

The Raspberry Pi acts as the central processing unit of the system. It receives input from the USB camera for real-time data capture and utilizes the memory card for storing datasets, trained models, and system files. A regulated power supply ensures proper functioning of all components.

The captured input data is processed by the Raspberry Pi using deep learning models deployed on the device. Based on the analysis, the system classifies the media as real or fake. The final result is displayed on the LCD screen, and a buzzer alert is triggered whenever fake content is detected.

System Components Description:

1. Power Supply:

   Provides stable power to operate the Raspberry Pi and connected devices.

2. Raspberry Pi:

   Acts as the main processing unit that performs deepfake detection.

3. USB Web Camera:

   Captures real-time image or video input.

4. Memory Card:

   Stores datasets, trained models, and system files.

5. Processing (Deep Learning Model):

   Analyzes input data and classifies it as real or fake.

6. LCD Display:

   Displays the detection result.

7. Buzzer:

   Alerts the user when fake media is detected.

4.6 System Workflow

The system workflow describes the sequence of operations performed by the proposed deepfake detection system during multimedia analysis.

The workflow begins when the user provides the path of the multimedia file to the system. The system then processes the input media using appropriate preprocessing techniques and deep learning models to determine whether the content is real or manipulated.

The proposed system uses separate deep learning models for different types of media, including images, videos, and audio recordings. MobileNetV2 is used for image and video deepfake detection, while the LCNN model is used for detecting manipulated audio signals.

The final classification result is displayed to the user through the monitor and LCD display, and an alert is generated when fake media is detected.

The complete workflow of the system is illustrated in **Fig 4.3**.

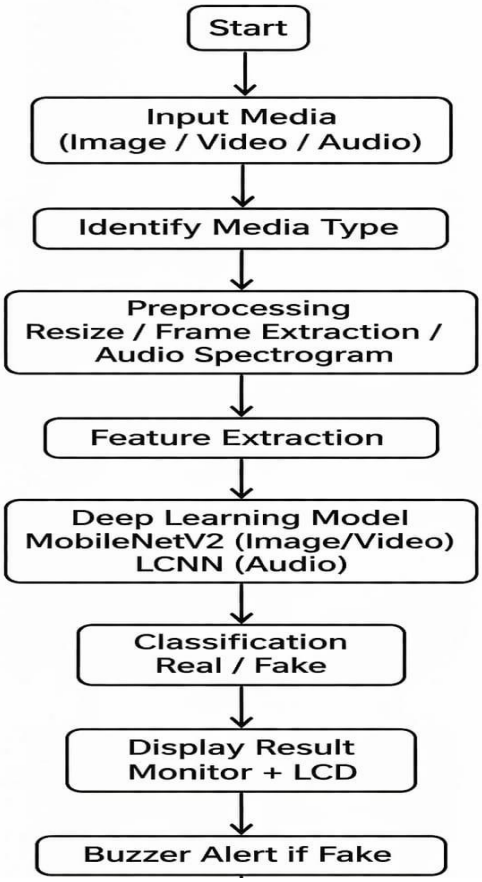

Fig 4.3 System Workflow of the Proposed System

Steps in the System Workflow

The operation of the proposed system can be explained through the following steps:

Step 1: Media Input

The workflow begins when the user provides the path of the multimedia file to the system. The input media may be one of the following types:

1. Image file
2. Video file
3. Audio file

The user copies the file path and provides it to the program through the Python interface.

Step 2: Media Loading

After receiving the input path, the system loads the corresponding multimedia file from the storage device. The file is read into the system memory for further processing and analysis.

Step 3: Media Preprocessing

Before performing deepfake detection, the input media undergoes preprocessing operations to standardize the data format and improve detection accuracy.

The preprocessing techniques depend on the type of input media.

For **image inputs**, the preprocessing operations include:

1. Loading the image file
2. Resizing the image to a fixed size of **224 × 224 pixels**
3. Normalizing the pixel values

For **video inputs**, the preprocessing operations include:

1. Loading the video file
2. Extracting individual frames using OpenCV
3. Resizing each frame to **224 × 224 pixels**

For **audio inputs**, the preprocessing operations include:

1. Loading the audio signal
2. Converting the audio signal into a **Mel spectrogram**
3. Resizing the spectrogram to **224 × 224 resolution**

These preprocessing steps ensure that the input data matches the format required by the deep learning models.

Step 4: Feature Extraction

After preprocessing, relevant features are extracted from the multimedia data. Feature extraction helps the system identify patterns that distinguish real media from deepfake manipulations.

For image and video inputs, the convolutional neural network automatically extracts spatial features such as:

1. Texture inconsistencies
2. Facial feature distortions
3. Pixel-level artifacts

For audio inputs, the spectrogram representation allows the model to detect irregular frequency patterns that indicate synthetic speech.

Step 5: Deep Learning Model Processing

The processed data is then passed to the trained deep learning models for classification. The proposed system uses the following models:

1. **MobileNetV2 Model**

MobileNetV2 is used for detecting deepfake manipulations in images and video frames. The model analyzes spatial patterns and visual artifacts to determine whether the media has been manipulated.

2. **LCNN Model**

The LCNN (Light Convolutional Neural Network) model is used for detecting manipulated audio signals. The model analyzes spectrogram features of the audio signal to identify synthetic speech patterns.

Each model processes the input media and generates a prediction indicating whether the content is real or fake.

Step 6: Media Classification

Based on the analysis performed by the deep learning models, the system classifies the input media into one of two categories:

1. Real Media

2. Fake Media

The classification is generated using the final output layer of the neural network model.

Step 7: Result Display

Once the classification process is completed, the detection result is displayed to the user. The output is shown through the following interfaces:

1. Python output window on the monitor

2. LCD display connected to the Raspberry Pi

The result clearly indicates whether the analyzed media is real or fake.

Step 8: Alert Generation

If the system detects manipulated media, a buzzer alert is activated. The buzzer generates an audible signal to notify the user that fake media has been detected.This alert mechanism allows users to quickly identify suspicious multimedia content.

4.7 Deepfake Detection Algorithm

The deepfake detection algorithm describes the sequence of operations performed by the system during multimedia analysis.

Algorithm: Multi-Modal Deepfake Detection

Step 1: Initialize the Raspberry Pi system.
Step 2: Load the trained TensorFlow Lite models.
Step 3: Accept multimedia input from the user.
Step 4: Load the input media.
Step 5: Apply appropriate preprocessing techniques.
Step 6: Extract features from the input data.
Step 7: Pass the features to the deep learning model.
Step 8: Perform deepfake detection using the trained model.
Step 9: Classify the input media as Real or Fake.
Step 10: Display the detection result on the monitor.
Step 11: Display the detection result on the LCD screen.
Step 12: Activate the buzzer if fake media is detected.
Step 13: End the process.

4.8 Dataset Description

The deep learning models used in this project are trained using datasets containing both real and manipulated multimedia samples.

These datasets help the model learn patterns associated with deepfake manipulations.

The datasets used in this project are collected from publicly available sources such as Kaggle. Due to the prototype nature of the system, a limited number of samples from each dataset are used for training and testing.

The dataset consists of three main categories:

1. Image Dataset
2. Video Dataset
3. Audio Dataset

These datasets are organized into separate folders containing real and fake samples.

4.8.1 Image Dataset Structure

Table 4.1 Image Dataset Used in the Proposed System

Category	Description
Real Images	Original face images collected from datasets
Fake Images	Deepfake manipulated face images

4.8.2 Video Dataset Structure

Table 4.2 Video Dataset Used in the Proposed System

Category	Description
Real Videos	Authentic video clips containing natural facial movements
Fake Videos	Deepfake manipulated videos with altered faces

4.8.3 Audio Dataset Structure

Table 4.3 Audio Dataset Used in the Proposed System

Category	Description
Real Audio	Genuine human voice recordings
Fake Audio	AI-generated or manipulated speech samples

The datasets used in this project were collected from publicly available sources such as Kaggle. A limited number of samples from each dataset were selected for training and testing due to the prototype nature of the system.

4.9 Deepfake Detection Models

The proposed multi-modal deepfake detection system utilized deep learning models to identify manipulated multimedia content. Different models were used for different media types in order to improve detection accuracy and computational efficiency.

The system implemented two major deep learning architectures:

1. **MobileNetV2** for image deepfake detection and video frame analysis.
2. **Light Convolutional Neural Network (LCNN)** for audio deepfake detection.

These models were selected because they provide a good balance between **high detection accuracy and low computational complexity**, which makes them suitable for deployment on embedded devices such as the **Raspberry Pi**.

4.9.1 MobileNetV2 Model for Image Detection

MobileNetV2 is a lightweight convolutional neural network architecture designed for efficient image classification tasks. It was developed specifically for mobile and embedded devices where computational resources are limited.

In this project, the MobileNetV2 model was used to detect deepfake manipulations in facial images. The model was trained using real and manipulated images obtained from publicly available deepfake datasets.

The architecture of MobileNetV2 consists of multiple convolution layers and inverted residual blocks that allow efficient feature extraction while reducing the number of parameters.

Working Principle

The working of the MobileNetV2 image detection model involved several processing stages, as shown in **Figure 4.4.**

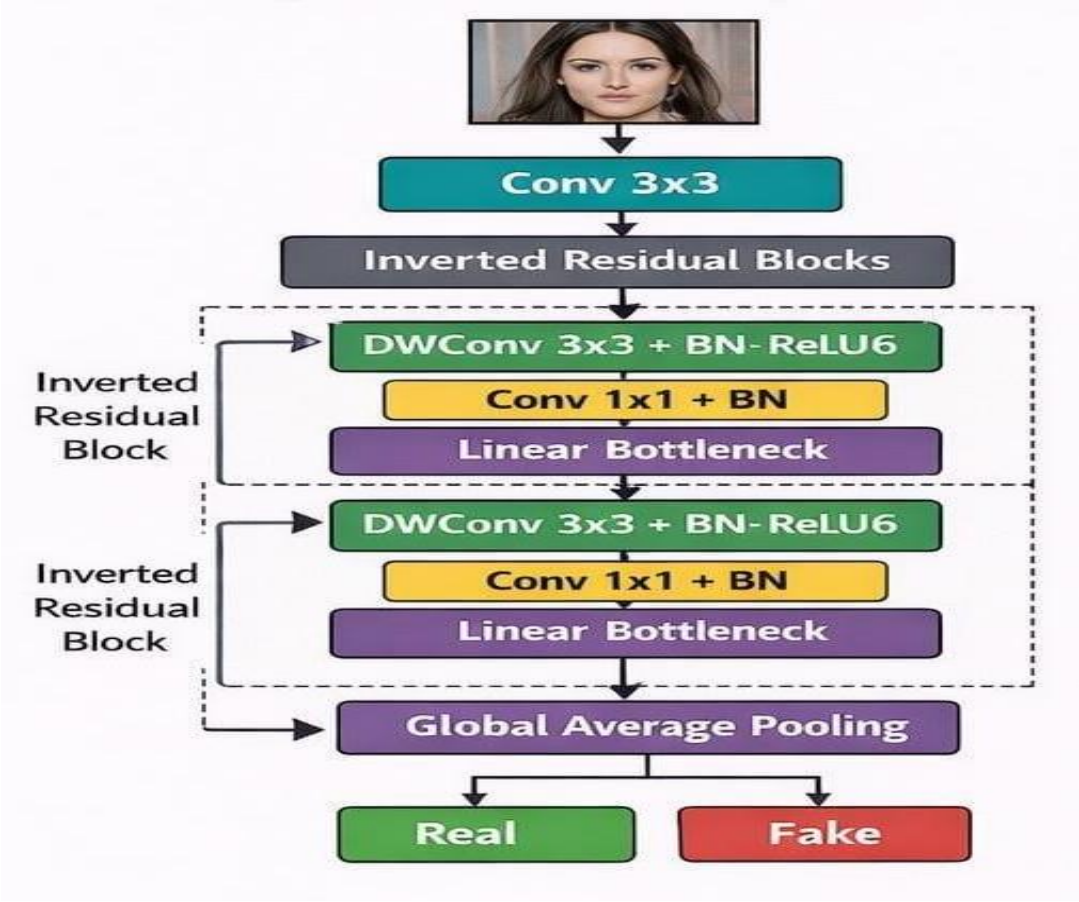

Fig 4.4: Architecture of MobileNetV2

First, the input image was provided to the system. The image was then resized to a fixed dimension so that it could be processed by the neural network.

After preprocessing, the image passed through several convolution layers that extracted important visual features such as edges, textures, and facial patterns.

The extracted features were then processed through **inverted residual blocks**, which are the key components of the MobileNetV2 architecture. These blocks use **depthwise separable convolution layers(DWConv)** that significantly reduce the number of computations required.

The inverted residual blocks consist of three main steps:

1. **Depthwise convolution layer** that filters spatial information from the input feature maps.
2. **Pointwise convolution layer** that combines the extracted features.
3. **Linear bottleneck layer** that reduces the dimensionality of the feature maps.

These blocks enable the network to learn important facial patterns that help identify inconsistencies caused by deepfake manipulations.

After passing through several inverted residual blocks, the extracted features were processed by a **global average pooling layer**. This layer reduces the spatial dimensions of the feature maps and converts them into a feature vector.

Finally, the feature vector was passed to a **fully connected classification layer**, which predicted whether the input image was **Real** or **Fake**.

Advantages of MobileNetV2

The MobileNetV2 architecture was selected for the following reasons:

1. It is computationally efficient and suitable for edge devices.
2. It uses depthwise separable convolutions that reduce processing cost.
3. It provides good feature extraction capability for image analysis.

4. It supports real-time inference when deployed using TensorFlow Lite.

Due to these advantages, MobileNetV2 was well suited for implementing deepfake detection on the Raspberry Pi platform.

4.9.2 MobileNetV2 Model for Video Frame Detection

Deepfake manipulation frequently occurs in video content where the face of a person is replaced or altered across multiple frames. Detecting such manipulations requires analyzing individual frames extracted from the video.

In this project, video deepfake detection was performed using the same MobileNetV2 architecture used for image detection. Instead of analyzing the entire video directly, the system first extracted individual frames from the input video.

These frames were then processed using the MobileNetV2 model to determine whether the visual content was real or manipulated.

Video Processing Approach

The video detection process involved the following steps:

1. The input video file was provided to the system.
2. The video was divided into multiple frames using frame extraction techniques.
3. Each frame was resized and normalized during preprocessing.
4. The frames were passed through the MobileNetV2 network.
5. The model extracted spatial features from each frame.
6. The classification layer predicted whether the frame was real or fake.

By analyzing multiple frames in a video sequence, the system was able to identify visual inconsistencies that commonly appear in deepfake videos.

Feature Extraction

During video frame analysis, the MobileNetV2 model extracted several important visual features including:

- Facial texture variations
- Lighting inconsistencies
- Edge artifacts around facial boundaries
- Abnormal blending of facial regions

These features helped the system identify subtle manipulation artifacts present in deepfake videos.

Output Generation

After processing the video frames, the system generated a final classification result indicating whether the video contained manipulated content.

The detection result was then displayed through:

1. The Python output window on the monitor
2. The LCD display connected to the Raspberry Pi

If fake content was detected, the system also activated the buzzer alert mechanism.

4.9.3 LCNN Model for Audio Deepfake Detection

Deepfake technology is not limited to visual media. Modern deep learning techniques can also generate highly realistic synthetic speech that mimics a person's voice. Detecting such audio deepfakes requires analyzing subtle variations in the speech signal.

To address this problem, the proposed system uses a **Light Convolutional Neural Network (LCNN)** for detecting manipulated audio signals, as shown in **Figure 4.5**.

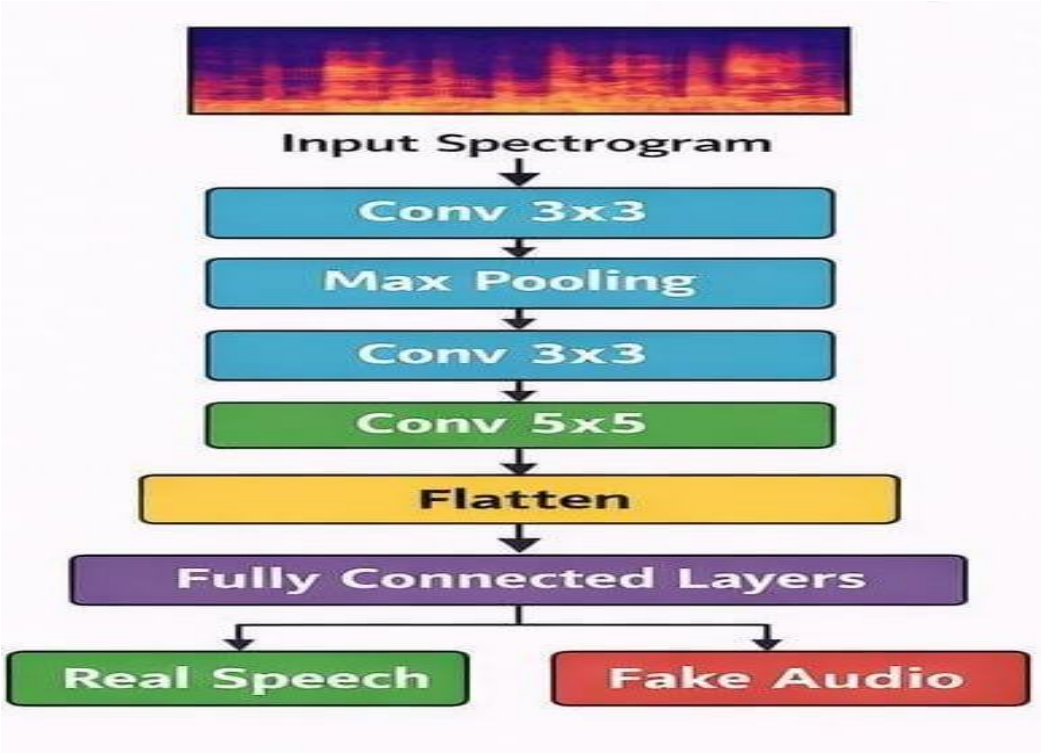

Fig 4.5: LCNN Architecture for Audio Deepfake Detection

Before the audio data could be processed by the LCNN model, it underwent several preprocessing steps.

First, the audio signal was converted into a visual representation known as a **Mel spectrogram**. A Mel spectrogram represents the frequency distribution of an audio signal over time and provides useful information for deep learning models.

The preprocessing stage involved the following steps:

1. Loading the audio signal.
2. Removing background noise if present.
3. Converting the audio waveform into a Mel spectrogram.
4. Normalizing the spectrogram values.

The generated spectrogram was then used as input to the LCNN model.

LCNN Architecture

The LCNN architecture consisted of several convolution layers and pooling layers that extracted spectral patterns from the input spectrogram.

The main components of the LCNN architecture included:

1. Convolution layers that extracted local frequency patterns.
2. Max pooling layers that reduced the spatial size of feature maps.
3. Additional convolution layers that captured deeper spectral features.
4. Flatten layer that converted feature maps into a one-dimensional vector.
5. Fully connected layers that performed classification.

These layers allowed the LCNN model to detect abnormal patterns that may indicate synthetic or manipulated speech.

Classification Process

After the audio spectrogram passed through the convolution layers, the extracted features were processed by fully connected layers.

The final classification layer determined whether the input audio signal was:

- **Real Speech**
- **Fake Audio**

The classification result was then displayed on the monitor and LCD screen.

Advantages of LCNN

The LCNN model was selected for audio deepfake detection because:

1. It effectively analyzes spectral patterns in audio signals.
2. It provides high detection accuracy for synthetic speech.
3. It requires fewer computational resources compared to large deep learning models.
4. It can be integrated with embedded systems for real-time audio analysis.

Because of these advantages, the LCNN architecture was suitable for implementing audio deepfake detection in the proposed multi-modal system.

4.10 Advantages of the Proposed System

The proposed deepfake detection system provides several advantages compared to traditional detection methods.

1. Supports multi-modal deepfake detection across images, videos, and audio.
2. Uses lightweight deep learning models suitable for embedded devices.
3. Enables real-time detection of manipulated multimedia content.
4. Operates efficiently on Raspberry Pi edge computing platform.
5. Provides automatic alert mechanism using LCD display and buzzer.
6. Offers a portable and cost-effective deepfake detection solution.

4.11 Applications of the Proposed System

The proposed system can be applied in several domains where multimedia authenticity is critical.

1. Social media content verification
2. Digital media authentication
3. Cybersecurity monitoring systems
4. Online news verification
5. Fraud detection systems
6. Multimedia forensic analysis

By detecting manipulated multimedia content, the system helps improve trust and reliability in digital communication platforms.

CHAPTER 5

HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Introduction

The development of the proposed Raspberry Pi–based multi-modal deepfake detection system required a combination of hardware components and software tools. The hardware components provided the physical infrastructure necessary for data processing, input acquisition, and result display. Meanwhile, the software environment enabled the implementation of deep learning models and multimedia processing algorithms required for detecting deepfake content.

The system was designed to operate as an edge computing platform capable of performing multimedia analysis directly on a Raspberry Pi device. The hardware setup consisted of a Raspberry Pi board, USB webcam, LCD display, buzzer module, power supply, and memory card. These components worked together to acquire multimedia inputs, process data using trained deep learning models, and present detection results to the user.

The software environment included Python programming language, Raspberry Pi OS, TensorFlow Lite, OpenCV, and supporting network utilities. These tools enabled model deployment, media processing, and system communication. The deep learning models were trained on a laptop and then converted into TensorFlow Lite format for efficient inference on the Raspberry Pi device.

This chapter provides a detailed explanation of the hardware components and software tools used in the project, including their specifications, functionalities, installation procedures, and their role in the deepfake detection system.

5.2 Hardware Components

The hardware components formed the physical platform required to implement the deepfake detection system. These components were responsible for multimedia input acquisition, data processing, and output presentation.

The major hardware components used in the project included:

1. Raspberry Pi 4 Model B
2. USB Webcam
3. LCD Display
4. Buzzer Module
5. Memory Card
6. Power Supply Adapter

5.2.1 Raspberry Pi 4 Model B

The Raspberry Pi 4 Model B served as the central processing unit of the deepfake detection system, as shown in **Figure 5.1**. It is a compact single-board computer capable of performing complex computational tasks while consuming relatively low power. The Raspberry Pi platform was selected because of its affordability, flexibility, and compatibility with artificial intelligence frameworks such as TensorFlow Lite.

In the proposed system, the Raspberry Pi performed the following tasks:

- Executing Python programs for multimedia processing
- Loading and running trained deep learning models
- Performing inference using TensorFlow Lite
- Processing multimedia inputs such as images, videos, and audio files
- Displaying detection results on the monitor and LCD screen
- Controlling peripheral devices such as the buzzer

The Raspberry Pi operated as an edge computing device, meaning that all processing was performed locally without requiring cloud-based computation.

Specifications of Raspberry Pi 4 Model B:

The major specifications of the Raspberry Pi 4 Model B include

- Quad-core ARM Cortex-A72 processor
- Processor speed up to 1.5 GHz
- 2GB / 4GB / 8GB RAM variants
- Dual HDMI display output
- USB 3.0 and USB 2.0 ports
- Gigabit Ethernet connectivity
- Built-in Wi-Fi and Bluetooth support
- MicroSD card slot for storage

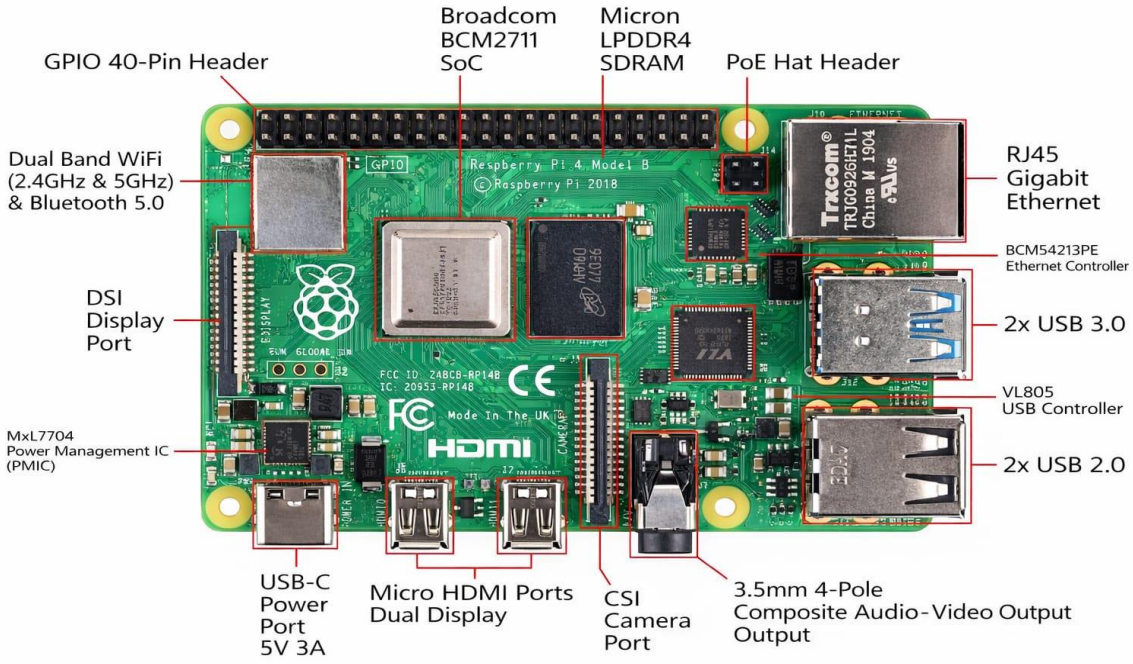

Fig 5.1: Raspberry Pi 4 Model B

These features make the Raspberry Pi capable of supporting deep learning inference tasks while maintaining low power consumption.

Role in the Project:

In this project, the Raspberry Pi performed several key operations:

1. Running the deepfake detection application developed in Python.
2. Loading the trained TensorFlow Lite models.
3. Processing multimedia inputs provided by the user.
4. Performing classification using deep learning models.
5. Displaying the output results on the connected display devices.

The Raspberry Pi effectively acted as the control unit that coordinated all system operations.

5.2.2 USB Webcam

The USB webcam was used as a multimedia input device for capturing real-time images and video streams, as shown in **Figure 5.2**. It provided visual data that could be analyzed by the deepfake detection models.

Fig 5.2: USB Webcam used for capturing image

The webcam connected to the Raspberry Pi through the USB interface and was accessed using the OpenCV library within the Python environment.

Functions of the USB Webcam:

The webcam supported the following functionalities in the system

- Capturing live images for analysis
- Recording video streams for deepfake detection
- Providing real-time visual input to the system

The captured frames were processed using the deep learning models to determine whether the content was authentic or manipulated.

Role in the Project:

In this project, the webcam was used to demonstrate the real-time capability of the system. The camera captured frames that were processed by the MobileNetV2 model to analyze facial features and detect potential deepfake manipulations.

5.2.3 LCD Display

The LCD display was used to present the detection results generated by the system, as illustrated in **Figure 5.3**. It provided a simple visual interface for users to view whether the analyzed multimedia content was real or fake.

The LCD module was connected to the Raspberry Pi through the GPIO interface using an I2C communication module.

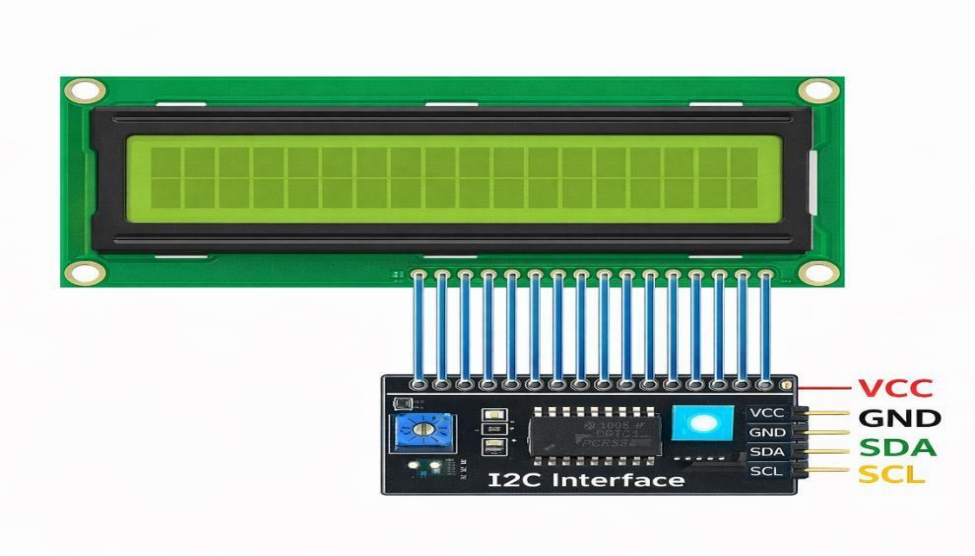

Fig 5.3: 16×2 LCD Display

Functions of LCD Display:

When the system processed multimedia input, the classification result was printed on both the monitor screen and the LCD display. This ensured that the output could be viewed even without direct access to the monitor interface.

5.2.4 Buzzer Module

The buzzer module served as an alert mechanism in the system, as shown in **Figure 5.4**. It produced an audible signal whenever the system detected manipulated or fake multimedia content.

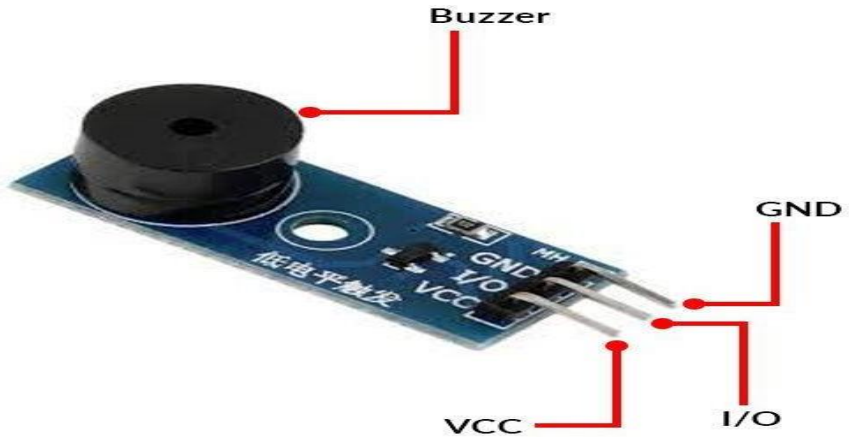

Fig 5.4: Buzzer

The buzzer was connected to one of the GPIO pins of the Raspberry Pi and was controlled through Python scripts.

Functions of the Buzzer:

The buzzer performed the following tasks:

- Generated an audible alert when fake media was detected
- Provided immediate feedback to the user

Role in the Project:

Whenever the deepfake detection model classified the input as fake, the Raspberry Pi triggered the buzzer module. This alert mechanism helped notify the user instantly.

5.2.5 Memory Card

The memory card served as the primary storage device for the Raspberry Pi system. It stored the operating system, project files, trained models, and datasets required for executing the application.

Functions of Memory Card:

The memory card stored the following data:

- Raspberry Pi operating system
- Python scripts for deepfake detection
- TensorFlow Lite models
- Dataset samples used for testing
- System configuration files

5.2.6 Power Supply Adapter

The Raspberry Pi required a stable power supply for proper operation. A 5V 3A USB-C power adapter was used to provide sufficient electrical power to the device.

Functions of the Power Adapter:

- Stable power delivery to the Raspberry Pi
- Reliable system performance
- Protection against voltage fluctuations

A stable power supply is essential for ensuring uninterrupted execution of the deep learning models and system processes.

5.3 Software Components

The software tools used in the project enabled the implementation of artificial intelligence algorithms, multimedia processing, and system communication.

The major software components used in the system included:

1. Raspberry Pi OS
2. Python Programming Language
3. OpenCV Library
4. TensorFlow Lite Framework
5. VNC Viewer
6. Advanced IP Scanner

These tools provided the necessary environment for developing and deploying the deepfake detection system.

5.3.1 Raspberry Pi OS

Raspberry Pi OS is the official operating system designed for Raspberry Pi devices. It is based on the Linux operating system and provides a stable platform for running applications.

Role in the Project:

Raspberry Pi OS was used to:

- Manage system resources
- Execute Python scripts
- Support AI libraries and dependencies
- Control hardware peripherals

The operating system also provided networking capabilities that allowed remote access to the Raspberry Pi device.

5.3.2 Python Programming Language

Python served as the primary programming language used to develop the deepfake detection system.

Python was chosen because it provides strong support for artificial intelligence libraries and multimedia processing frameworks.

Role of Python in the Project:
Python was used to:

- Implement deepfake detection algorithms
- Load trained machine learning models
- Process multimedia inputs
- Control hardware components
- Display classification results

The flexibility of Python made it suitable for integrating hardware and software functionalities.

5.3.3 OpenCV Library

OpenCV (Open Source Computer Vision Library) is a widely used library for image and video processing applications.

In this project, OpenCV was used to perform several multimedia processing tasks.

Functions of OpenCV:

OpenCV enabled the system to perform:

- Image loading and preprocessing
- Video frame extraction
- Camera input capture
- Image resizing and normalization

These operations prepared the multimedia data before it was provided to the deep learning models.

5.3.4 TensorFlow Lite Framework

TensorFlow Lite is a lightweight version of the TensorFlow machine learning framework designed for embedded and mobile devices.

In this project, deep learning models were trained on a laptop using TensorFlow and then converted into TensorFlow Lite format for efficient execution on the Raspberry Pi.

Advantages of TensorFlow Lite:

TensorFlow Lite provides several benefits:

- Reduced model size
- Faster inference speed
- Lower memory usage

Role in the Project:

TensorFlow Lite was used to perform the final deepfake classification on the Raspberry Pi device.

5.3.5 VNC Viewer

VNC Viewer is a remote desktop software that allows users to access the graphical interface of the Raspberry Pi from another computer.

Role in the Project:

VNC Viewer was used to:

- Access the Raspberry Pi desktop remotely
- Run Python scripts from a laptop
- Monitor system outputs

This feature simplified system development and debugging.

5.3.6 Advanced IP Scanner

Advanced IP Scanner is a network utility used to detect devices connected to the same local network.

Role in the Project:

The tool helped identify the IP address of the Raspberry Pi device, which was required for establishing remote connections using VNC Viewer.

5.3.7 Software Implementation Workflow

The software components worked together to enable the execution of the deepfake detection system on the Raspberry Pi platform. The implementation involved integrating Python programming with multimedia processing libraries and deep learning frameworks.

The workflow of the software implementation involved the following steps:

1. The Raspberry Pi OS provided the operating system environment for running the application.
2. Python scripts were developed to control system operations and implement the detection algorithms.
3. OpenCV was used to capture and preprocess multimedia inputs such as images

and video frames.

4. The trained deep learning models were loaded using the TensorFlow Lite interpreter.
5. The input media was processed by the deep learning models to classify it as real or fake.
6. The classification result was displayed on the monitor and LCD display.
7. If fake media was detected, the buzzer alert mechanism was activated.

This workflow enabled efficient interaction between the hardware components and software tools used in the system.

5.3.8 Model Deployment on Raspberry Pi

The deep learning models used in the project were initially trained on a laptop environment using larger computational resources. After the training process was completed, the models were converted into TensorFlow Lite format to enable efficient deployment on the Raspberry Pi device.

The deployment process involved the following steps:

1. Training the deep learning models using image, video, and audio datasets.
2. Converting the trained models into TensorFlow Lite (.tflite) format.
3. Transferring the converted model files to the Raspberry Pi system.
4. Loading the models using the TensorFlow Lite interpreter in Python.
5. Performing inference on multimedia inputs to detect deepfake content.

This approach allowed the system to perform deepfake detection directly on the Raspberry Pi without requiring high-performance computing resources.

CHAPTER 6

EXPERIMENTAL RESULTS

6.1 Introduction

This chapter presents the experimental evaluation of the proposed multi-modal deepfake detection system. The system is designed to analyze images, videos, and audio inputs using deep learning models deployed on a Raspberry Pi-based edge device.

The system architecture supports three detection modules:

- Image deepfake detection
- Video deepfake detection
- Audio deepfake detection

At the current stage, the video detection module has been fully implemented and tested. The image and audio modules have been developed and integrated at the model level, and their experimental results will be presented in future work.

6.2 Experimental Results and Output Analysis

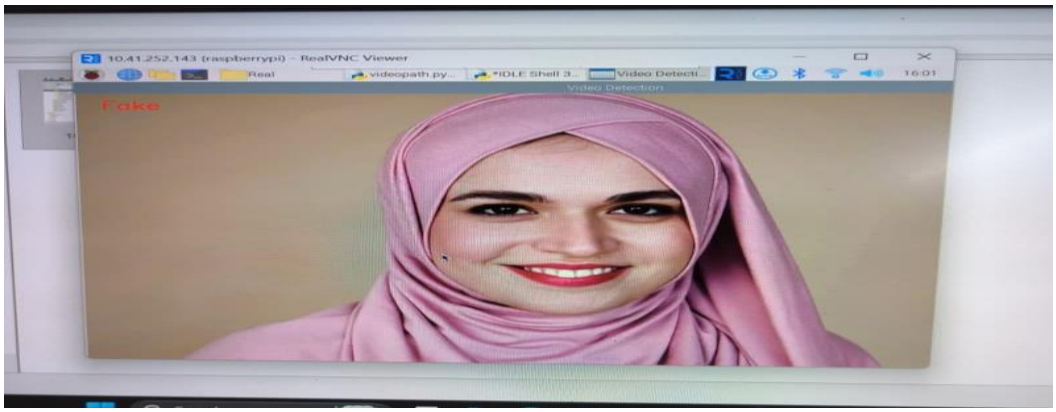

Figure 6.1: Detection of Fake Video Frame

Figure 6.1 shows a video frame that has been classified as “Fake” by the proposed deepfake detection system. The model analyzes the input frame by resizing and normalizing it before passing it through the trained convolutional neural network. Based on learned features such as facial inconsistencies, unnatural texture patterns, and blending

artifacts, the system predicts the frame as fake. The output label is displayed on the screen in real time, indicating successful frame-level classification during video processing.

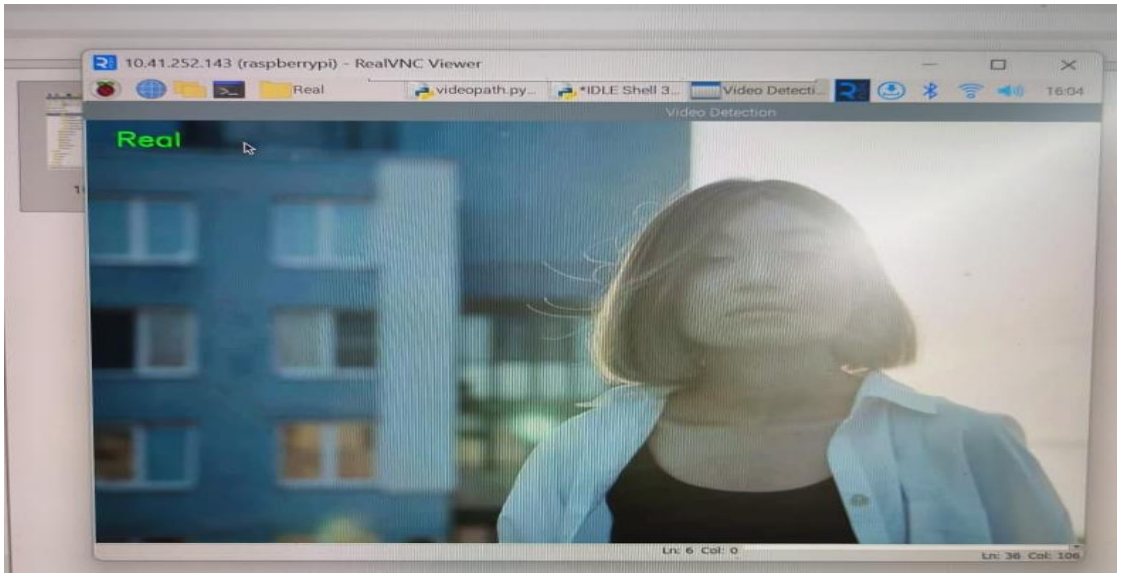

Figure 6.2: Detection of Real Video Frame

Figure 6.2 illustrates a video frame classified as “Real” by the deepfake detection system. The model processes the frame by resizing and normalizing it before performing inference using the trained convolutional neural network. Based on features such as consistent facial structure, natural lighting, and absence of visual artifacts, the system predicts the frame as real. The result is displayed in real time, demonstrating accurate frame-level classification during video analysis.

Figure 6.3: LCD Display Showing Final Detection Output

Figure 6.3 shows the LCD display presenting the final detection output as “REAL”. This confirms that the system has classified the processed video as authentic after evaluating multiple frames. This verifies the correct functioning of the hardware interface and real-time communication between the Raspberry Pi and the display module. It also highlights the system’s capability to provide immediate and clear user feedback.

To further validate the proposed system, additional outputs from image-level and audio-based detection are presented. These results demonstrate the system’s capability to handle multi-modal inputs while maintaining consistent classification performance in real-time.

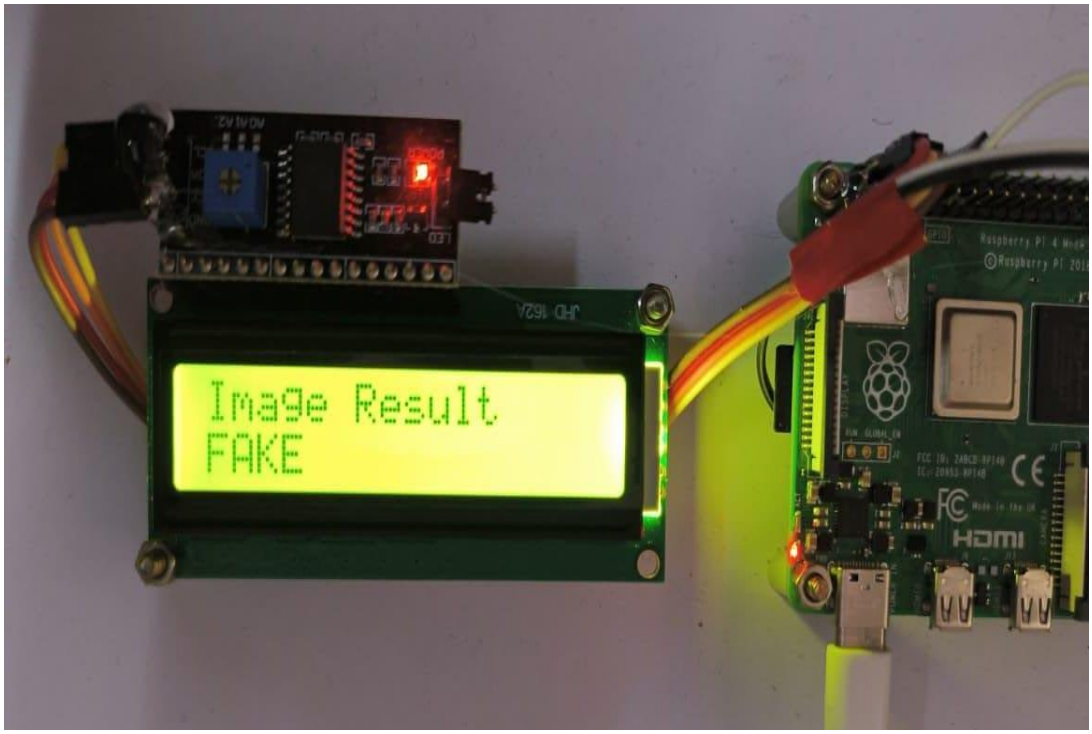

Figure 6.4: LCD Output Display Showing Real Classification

Figure 6.4 shows the LCD display presenting the classification result as “FAKE”. This indicates that the system has detected manipulated content and successfully identified it as fake.

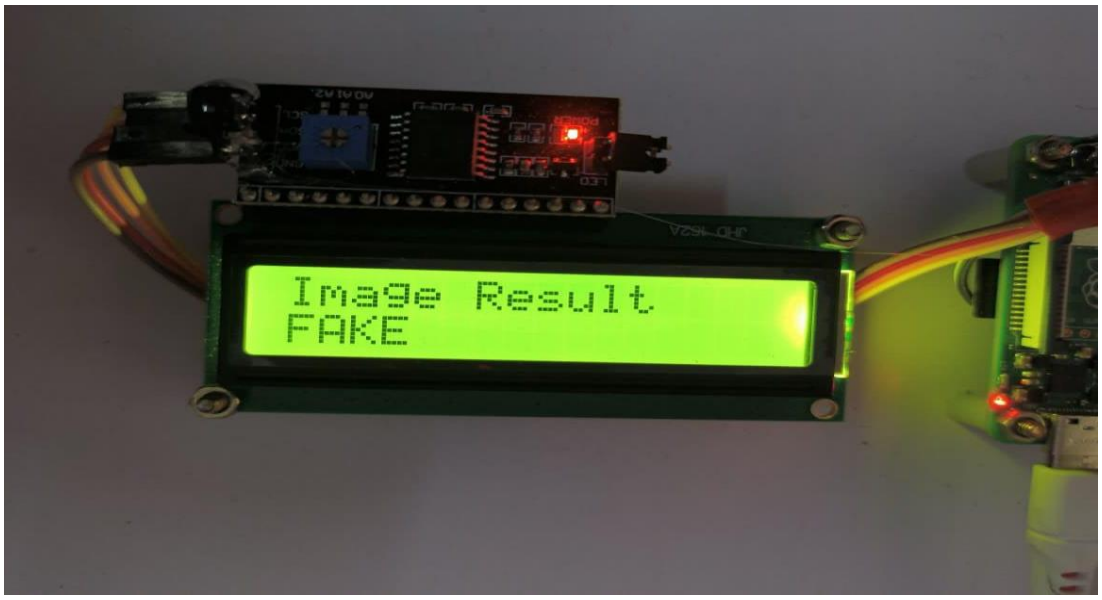

Figure 6.5: LCD Output Display Showing Fake Classification

Figure 6.5 illustrates the LCD output displaying "FAKE" for manipulated input data. The result demonstrates the system's ability to detect inconsistencies and classify the input as fake in real time.

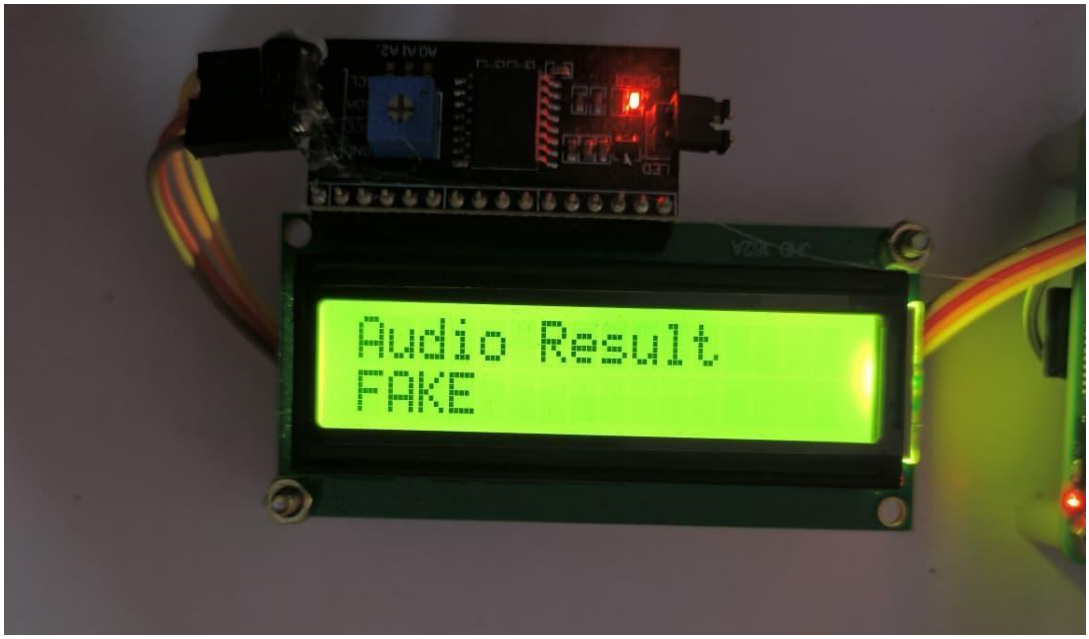

Figure 6.6: Audio-Based Detection Result – Fake Output

Figure 6.6 shows the audio-based detection result where the input audio is classified as "FAKE." The system analyzes features such as frequency variations and voice inconsistencies to identify synthetic or manipulated audio signals.

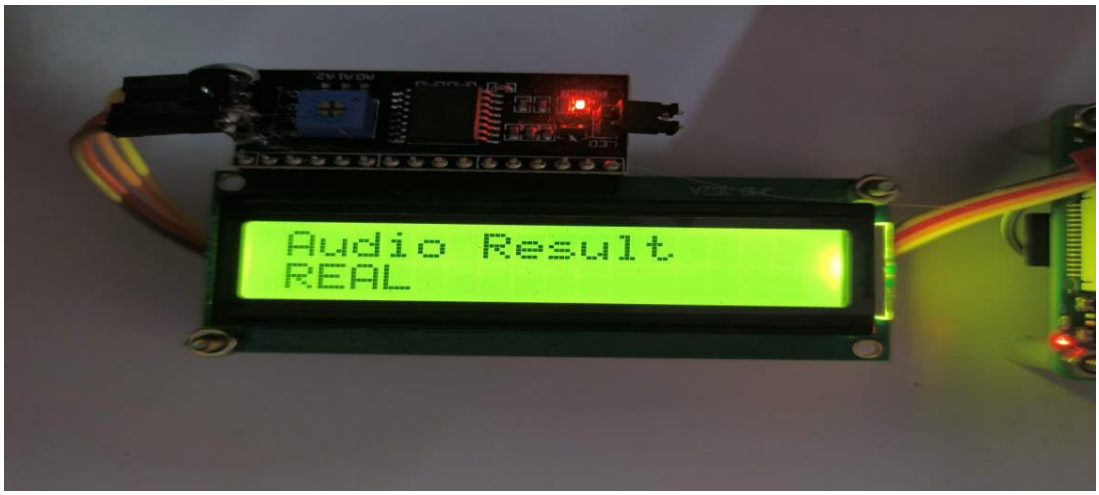

Figure 6.7: Audio-Based Detection Result – Real Output

Figure 6.7 illustrates the classification of a genuine audio sample as "REAL." The system verifies natural speech patterns and consistency in the audio signal before confirming it as authentic.

6.3 Performance Evaluation Based on Classification Metrics

The performance of the proposed deepfake detection system is evaluated using standard classification metrics derived from the confusion matrix. These metrics provide a detailed understanding of how effectively the system distinguishes between real and fake media samples. The obtained results are summarized in Table 6.1.

Table 6.1: Performance Metrics of Proposed System

Metric	Performance (%)
Accuracy	94.2%
Precision	93.6%
Recall	92.8%
F1-Score	93.2%

6.3.1 Evaluation Metrics Definition

The following metrics are used for performance evaluation:

- **Accuracy**

Measures the overall correctness of the model.

Formula:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

- **Precision**

Indicates how many predicted fake samples are actually fake.

Formula:

$$\text{Precision} = TP / (TP + FP)$$

- **Recall**

Measures how many actual fake samples are correctly detected.

Formula:

$$\text{Recall} = TP / (TP + FN)$$

- **F1-Score**

Harmonic mean of precision and recall.

Formula:

$$\text{F1-Score} = (2 \times \text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Where:

- TP (True Positive) → Fake correctly detected
- TN (True Negative) → Real correctly detected
- FP (False Positive) → Real incorrectly detected as fake
- FN (False Negative) → Fake incorrectly detected as real

6.3.2 Computation of Results

The obtained performance values are calculated based on predictions made by the model on the test dataset.

- The dataset is divided into training (80%) and testing (20%)
- The trained model is evaluated using unseen test samples
- Predictions are compared with actual labels to construct a confusion matrix
- From the confusion matrix, values of TP, TN, FP, and FN are obtained
- These values are substituted into the metric formulas to compute the final performance percentages

6.3.3 Performance Analysis

The results presented in Table 5.X indicate that the proposed system achieves high performance across all evaluation metrics.

- **Accuracy (94.2%)**

Indicates that the system correctly classifies the majority of input samples, demonstrating strong overall performance.

- **Precision (93.6%)**

Shows that the system produces very few false positives, meaning predictions of fake content are highly reliable.

- **Recall (92.8%)**

Indicates that most of the fake samples are successfully detected, although a small number may still be missed.

- **F1-Score (93.2%)**

Represents a balance between precision and recall, confirming consistent and stable model performance.

6.3.4 Key Observations

- The system achieves high accuracy, indicating reliable classification performance
- High precision ensures low false alarm rates
- Balanced recall ensures effective detection of fake content
- The F1-score confirms overall model stability
- The system performs efficiently even on Raspberry Pi edge hardware
- Lightweight models such as MobileNetV2 enable real-time processing without significant loss in accuracy

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

The proposed project developed a Raspberry Pi–enabled multi-modal AI system for deepfake detection in multimedia, capable of identifying manipulated images, videos, and audio recordings. Lightweight deep learning models were utilized to ensure efficient operation on an edge computing platform. MobileNetV2 was implemented for image and video deepfake detection, while an LCNN model was used for audio deepfake detection. The trained models were converted into TensorFlow Lite format and deployed on a Raspberry Pi 4 device to enable real-time inference with minimal computational resources. The system successfully analyzed multimedia inputs and produced classification results indicating whether the content was real or fake. The results were displayed through the monitor and LCD interface, and a buzzer alert mechanism was triggered when manipulated media was detected. Experimental evaluation showed that the system achieved an overall detection accuracy of 94.2%, demonstrating that edge-based artificial intelligence systems can provide an efficient and portable solution for real-time deepfake detection.

FUTURE SCOPE

The proposed system can be further improved by training the models with larger and more diverse datasets to increase detection accuracy. Advanced deep learning models and multimodal fusion techniques can be integrated to enhance performance. The system can also be extended to support real-time monitoring of live multimedia streams and deployed in web or mobile applications for wider usage.

REFERENCES

[1] Eunji Kim and Sungzoon Cho, “Exposing Fake Faces with Content and Trace Features”, IEEE Transactions on Information Forensics and Security, Vol.16, No.4, Apr 2021, pp. 1281–1294.

[2] Md Shohel Rana, Nurul Mohammed and Md Zahid Hasan, “Deepfake Detection: A Systematic Literature Review”, IEEE Access, Vol.10, No.1, Jan 2022, pp. 25413–25438.

[3] Thanh Thi Nguyen, Cuong T. Nguyen, Dung Tien Nguyen, Duc Tien Nguyen, Saeid Nahavandi and Thanh Thi Nguyen, “Deep Learning for Deepfakes Creation and Detection: A Survey”, Computer Vision and Image Understanding, Vol.223, No.3, Aug 2022, pp. 103525–103540.

[4] Jerry John and Bismin Sherif, “Comparative Analysis of Deepfake Detection Methods”, International Journal of Computer Applications, Vol.183, No.45, Jul 2022, pp. 1–7.

[5] Amala Mary and Anitha Edison, “Deepfake Detection Using Deep Learning Techniques: A Review”, Journal of Artificial Intelligence and Data Mining, Vol.11, No.2, Mar 2023, pp. 315–327.

[6] Deepak Dagar and Vishwakarma, “Hybrid Xception-LSTM Model with Channel and Spatial Attention for Deepfake Detection”, IEEE Access, Vol.11, No.5, May 2023, pp. 40215–40228.

[7] Siva Rama Lingham, Praveen Kumar and Ramesh Kumar, “Development of Deepfake Detection Techniques for Protecting Multimedia Information”, International Journal of Multimedia Information Security, Vol.6, No.1, Jan 2024, pp. 12–22.

[8] Mudit Arya, Rohan Sharma and Ankit Verma, “Study on Deep Fake Face Detection Techniques”, International Journal of Computer Science and Artificial Intelligence, Vol.12, No.2, Feb 2024, pp. 95–104.

[9] Andreas Rossler, Davide Cozzolino, Luisa Verdoliva, Christian Riess, Justus Thies and Matthias Nießner, "FaceForensics++: Learning to Detect Manipulated Facial Images", IEEE International Conference on Computer Vision (ICCV), Oct 2019, pp. 1–11.

[10] Yuezun Li and Siwei Lyu, "Exposing DeepFake Videos by Detecting Face Warping Artifacts", IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), Jun 2019, pp. 46–52.

[11] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto and Hartwig Adam, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications", IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[12] Yevgeniy Lavrentyeva, Sergey Novoselov, Andrey Tseren and Alexander Kozlov, "Audio Replay Attack Detection with Deep Learning Frameworks", Interspeech Conference, Aug 2017, pp. 82–86.

[13] Kaede Shiohara and Toshihiko Yamasaki, "Detecting Deepfakes with Self-Blended Images", IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Jun 2022, pp. 18720–18729.

[14] Zhendong Wang, Qianli Ma, Jiahui Zheng, Zhenyu Wang and Jingjing Chen, "FakeCatcher: Detection of Synthetic Portrait Videos Using Biological Signals", IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol.45, No.3, Mar 2023, pp. 3304–3318.

[15] Ziyang Zhang, Jianqiang Huang and Shiguang Shan, "Multi-modal Deepfake Detection with Audio-Visual Consistency Learning", IEEE Transactions on Multimedia, Vol.26, 2024, pp. 1021–1034.

APPENDIX

PROJECT SOURCE CODE

The source code of the proposed system is divided into separate modules for handling different types of multimedia inputs.

• image_check.py → Processes image inputs

• video_check.py → Processes video inputs

• audio_check.py → Processes audio inputs

• lcd.py → Controls LCD display output

• threshold.py → Used for threshold tuning during model evaluation

Each module independently loads the trained TensorFlow Lite model and performs deepfake detection. The Raspberry Pi controls the execution and handles hardware outputs such as LCD display and buzzer alerts.

A.1 Image Deepfake Detection Code (image_check.py)

Description:

This program reads an input image, preprocesses it using resizing and normalization, and uses a TensorFlow Lite model to classify the image as real or fake. The result is displayed on an LCD, and a buzzer alert is triggered when fake content is detected.

Code:

```
# ==============================

# Image Deepfake Detection Code

# ==============================

import numpy as np

import cv2
```

```
import warnings

import tflite_runtime.interpreter as tflite

import RPi.GPIO as GPIO

import lcd

import time

warnings.filterwarnings("ignore")

# ==============================

# GPIO Setup

# ==============================

BUZZER = 18

GPIO.setmode(GPIO.BCM)

GPIO.setup(BUZZER, GPIO.OUT)

# ==============================

# Load TFLite Model

# ==============================

model_path = "/home/pi/Desktop/Raspberry PiBased MultiModal AI Edge System/image_model.tflite"

interpreter = tflite.Interpreter(model_path=model_path)

interpreter.allocate_tensors()

input_details = interpreter.get_input_details()

output_details = interpreter.get_output_details()
```

```
IMG_SIZE = 224

THRESHOLD = 0.5

# ==============================

# Image Path

# ==============================

image_path = "/home/pi/Desktop/Raspberry PiBased MultiModal AI Edge System/deepfake detection/Fake"

print("\nChecking Image File...\n")

lcd.display("Checking Image","")

try:

    img = cv2.imread(image_path)

    if img is None:

        raise Exception("Image not found")

    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))

    img = img / 255.0

    img = np.expand_dims(img, axis=0)

    img = img.astype(np.float32)

    interpreter.set_tensor(input_details[0]['index'], img)

    interpreter.invoke()

    pred = interpreter.get_tensor(output_details[0]['index'])[0][0]

    if pred >= THRESHOLD:
```

```
        label = "REAL IMAGE"
        confidence = pred * 100
        lcd.display("Image Result","REAL")
        GPIO.output(BUZZER, GPIO.LOW)
    else:
        label = "FAKE IMAGE"
        confidence = (1 - pred) * 100
        lcd.display("Image Result","FAKE")
        GPIO.output(BUZZER, GPIO.HIGH)
        print("FAKE IMAGE DETECTED")
        time.sleep(3)
        GPIO.output(BUZZER, GPIO.LOW)
    print("--------------------------------")
    print("Image File:", image_path)
    print("Raw Score:", round(float(pred),3))
    print("Prediction:", label)
    print("Confidence:", round(float(confidence),2), "%")
except Exception as e:
    lcd.display("Image Error","Check File")
    GPIO.output(BUZZER, GPIO.LOW)
    print("Error processing image:", e)
```

```
print("\nDetection Completed\n")

GPIO.cleanup()
```

A.2 Video Deepfake Detection Code (video_check.py)

Description:

Processes video frames and determines whether the video is real or fake using frame-wise classification.

Code:

```
# ==============================

# Video Deepfake Detection Code

# ==============================

import cv2

import numpy as np

import tflite_runtime.interpreter as tflite

import RPi.GPIO as GPIO

import lcd

import time

GPIO.setwarnings(False)

BUZZER = 18

GPIO.setmode(GPIO.BCM)

GPIO.setup(BUZZER, GPIO.OUT)

GPIO.output(BUZZER, GPIO.LOW)
```

```
model_path = "/home/pi/Desktop/Raspberry PiBased MultiModal AI Edge System/video_model.tflite"

interpreter = tflite.Interpreter(model_path=model_path)

interpreter.allocate_tensors()

input_details = interpreter.get_input_details()

output_details = interpreter.get_output_details()

IMG_SIZE = (224,224)

class_names = ["Fake","Real"]

video_path = "/home/pi/Desktop/Raspberry PiBased MultiModal AI Edge System/video dataset/Real/Real (1).mp4"

cap = cv2.VideoCapture(video_path)

fake_count = 0

real_count = 0

frame_id = 0

lcd.display("Video System","Processing")

print("\nVideo Detection Started\n")

while True:

    ret, frame = cap.read()

    if not ret:

        break

    if frame_id % 5 == 0:
```

```
        img = cv2.resize(frame, IMG_SIZE)
        img = img / 255.0
        img = np.expand_dims(img, axis=0).astype(np.float32)
        interpreter.set_tensor(input_details[0]['index'], img)
        interpreter.invoke()
        prediction = interpreter.get_tensor(output_details[0]['index'])[0]
        pred_class = np.argmax(prediction)
        label = class_names[pred_class]
        if label == "Fake":
            fake_count += 1
            color = (0,0,255)
        else:
            real_count += 1
            color = (0,255,0)
        cv2.putText(frame, label, (30,40),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)
    cv2.imshow("Video Detection", frame)
    if cv2.waitKey(1) & 0xFF == 27:
        break
    frame_id += 1
cap.release()
```

```
cv2.destroyAllWindows()

print("\nFake Frames:", fake_count)

print("Real Frames:", real_count)

if fake_count > real_count:

    print("Final Result: FAKE VIDEO")

    lcd.display("Video Result","FAKE")

    GPIO.output(BUZZER, GPIO.HIGH)

    time.sleep(3)

    GPIO.output(BUZZER, GPIO.LOW)

else:

    print("Final Result: REAL VIDEO")

    lcd.display("Video Result","REAL")

    GPIO.output(BUZZER, GPIO.LOW)

GPIO.cleanup()
```

A.3 Audio Deepfake Detection Code (audio_check.py)

Code:

```
# ==============================

# Audio Deepfake Detection Code

# ==============================

import numpy as np

import librosa
```

```
import cv2

import warnings

import tflite_runtime.interpreter as tflite

import RPi.GPIO as GPIO

import lcd

import time

warnings.filterwarnings("ignore")

BUZZER = 18

GPIO.setmode(GPIO.BCM)

GPIO.setup(BUZZER, GPIO.OUT)

model_path = "/home/pi/Desktop/Raspberry PiBased MultiModal AI Edge System/audio_model.tflite"

interpreter = tflite.Interpreter(model_path=model_path)

interpreter.allocate_tensors()

input_details = interpreter.get_input_details()

output_details = interpreter.get_output_details()

IMG_SIZE = 224

THRESHOLD = 0.87

audio_path = "/home/pi/Desktop/Raspberry PiBased MultiModal AI Edge System/filtered_audio/fake/Fake (1).wav"

print("\nChecking Audio File...\n")
```

```
lcd.display("Checking Audio","")
try:
    signal, sr = librosa.load(audio_path, sr=22050)
    mel = librosa.feature.melspectrogram(
        y=signal,
        sr=sr,
        n_mels=128
    )
    mel_db = librosa.power_to_db(mel, ref=np.max)
    mel_db = (mel_db - mel_db.min()) / (mel_db.max() - mel_db.min())
    mel_img = cv2.resize(mel_db, (IMG_SIZE, IMG_SIZE))
    mel_img = np.stack((mel_img,) * 3, axis=-1)
    mel_img = np.expand_dims(mel_img, axis=0)
    mel_img = mel_img.astype(np.float32)
    interpreter.set_tensor(input_details[0]['index'], mel_img)
    interpreter.invoke()
    pred = interpreter.get_tensor(output_details[0]['index'])[0][0]
    if pred >= THRESHOLD:
        label = "FAKE AUDIO"
        confidence = pred * 100
        lcd.display("Audio Result","FAKE")
```

```
        GPIO.output(BUZZER, GPIO.HIGH)
        print("FAKE AUDIO DETECTED")
        time.sleep(3)
        GPIO.output(BUZZER, GPIO.LOW)
    else:
        label = "REAL AUDIO"
        confidence = (1 - pred) * 100
        lcd.display("Audio Result","REAL")
        GPIO.output(BUZZER, GPIO.LOW)
    print("--------------------------------")
    print("Audio File:", audio_path)
    print("Raw Score:", round(float(pred),3))
    print("Prediction:", label)
    print("Confidence:", round(float(confidence),2), "%")
except Exception as e:
    lcd.display("Audio Error","Check File")
    GPIO.output(BUZZER, GPIO.LOW)
    print("Error processing audio:", e)
print("\nDetection Completed\n")
GPIO.cleanup()
```